

Python 基礎



WEB・プログラム・ネットワーク・OA・資格取得
KENスクール

目 次

1 章 Python とは	1
Python 言語の特徴	2
Python 言語が使われる用途	2
Python 言語のライブラリ	2
Python 言語の系列	2
Python の開発環境の構築	3
Anaconda Installers のダウンロード	3
Spyder の起動	3
Python プログラムの実行	4
Python プログラムの実行方法	4
インタラクティブ・シェルを試してみる	5
数値演算子	6
2 章 変数	9
変数とは	10
変数名のルール	10
文字列 (str)	11
文字列結合	12
f 文字列	12
[]で文字列を取り出す	13
文字列のスライス	14
コメント	15
整数型 (int) と浮動小数型 (float)	15
ブール型 (bool)	16
累算代入演算子	16
演算子の優先順位	17
型変換	17
型を調べる	18
3 章 条件分岐	19
if 文	20
比較演算子	20
if～else 文	21
if～elif～else 文	22
論理演算子	22

論理積 (and)	23
論理和 (OR)	23
否定 (not)	24
if 文のネスト	24
4 章 繰り返し処理.....	25
while 文	26
無限ループ	27
break 文.....	27
continue 文	28
多重ループ（繰り返しのネスト）	29
5 章 データ構造.....	31
データ構造とは何か.....	32
リスト (list)	32
リストへのデータの追加 : append()メソッド.....	33
リストへのデータの挿入 : insert()メソッド.....	33
リストからのデータの削除 : remove()メソッド	34
リストからのデータの削除 : pop()メソッド	35
リストのスライス	36
リストのデータのクリア : clear()メソッド.....	36
for 文によるリストの繰り返し処理	37
for 文のその他の使い方	37
range()関数を利用した連番リストの作成	38
リスト (list) の内包表記	38
タプル (tuple)	40
タプルのアンパック.....	41
集合 (set)	43
セット (Set) の内包表記	44
辞書 (dict)	45
辞書への要素追加	45
辞書の要素削除 : del 文.....	46
辞書の複数要素の更新 : update()メソッド	46
for 文による辞書の繰り返し処理	47
辞書 (dict) の内包表記	48
6 章 関数	49
関数とは何か	50
関数の定義	50

関数の呼び出し	51
関数を呼び出す場合の注意点	52
可変長引数	52
7 章 ファイルを分割する	55
モジュール	56
モジュールを読み込む方法	56
パッケージ	58
パッケージの読み込み	58
8 章 オブジェクト指向	65
オブジェクト指向とは	66
オブジェクト指向の考え方	66
クラスの定義	67
複数のインスタンス	68
継承	70
9 章 Python のライブラリ	73
pip コマンド	74
Matplotlib パッケージ	75
折れ線グラフ	75
グラフを保存する	76
NumPy ライブラリ	79
NumPy の配列	79
NumPy の多次元配列	81
配列データの構造確認	81
次元の変換	82
要素の追加	83
要素の挿入	84
要素へのアクセスと値の更新	85
要素の抽出と並べ替え	88
配列の演算	90
Python 基礎研修課題	95
BMI 判定問題	96
くじ引き問題	97
RPG のバトルシーン問題	98
自作関数問題	99
Python 基礎研修課題解答例	101

BMI 判定問題 解答例	102
くじ引き問題 解答例	102
RPG のバトルシーン問題 解答例	103
自作関数問題 解答例	104
Appendix	105
クラウド型実行環境【Colaboratory】	106
ファイルの準備	107
コードセルの編集と実行	109
テキストセルの編集	110
外部ファイルの準備	111
ライブラリのインストール	111
GPU の利用	112
接続が切れた場合	112

1 章 Python とは

Python 言語の特徴

Python 言語とは汎用のプログラミング言語です。数あるプログラミング言語の中では、コードがシンプルで扱いやすく設計されているのが特徴で、プログラムを分かりやすく、少ない行数で書けるため、初心者にも学びやすいのが特徴です。

Python 言語が使われる用途

Python 言語は様々な用途で活用されます。主な用途は以下のようなものです。

- 人工知能・機械学習
- データ分析・統計処理
- 処理の自動化
- Web アプリケーションの作成

Python 言語のライブラリ

Python 言語には数多くのライブラリが存在します。ライブラリとはプログラムの機能をまとめた道具箱のようなものです。Python 言語で利用できる主なライブラリには以下のようなものがあります。

ライブラリ名	使用目的
NumPy	数値計算処理を行うライブラリ
Matplotlib	Numpy をベースにしたグラフ描画のためのライブラリ
SciPy	科学技術計算を行うライブラリ。NumPy をベースに開発されている
Pandas	データ分析のためのライブラリ
Scikit-learn	機械学習のためのライブラリ
Tensorflow	深層学習などの人工知能のライブラリ

Python 言語の系列

Python 言語には **2.x** 系と **3.x** 系の 2 系統のバージョンがあります。**2.x** 系と **3.x** 系は一部ソースコードの互換性がありません。

2.x 系は 2010 年にリリースされた 2.7 が最終バージョンになり、**2020** 年にサポートが終了しました。**3.x** 系はメジャーなライブラリはほとんど対応しており、継続的にバージョンアップされています。

そこで、本講座では **3.x** 系統を前提として講座を進めます。

Python の開発環境の構築

Anaconda Installers のダウンロード

オールインワンパッケージである **Anaconda** を使用します。ここには定番ライブラリは全て含まれており、公式インストーラーも含まれています。

該当プラットフォームを選択の上、**3.x** 系をダウンロードし、**64** ビット版か **32** ビット版かを選択してインストールします。

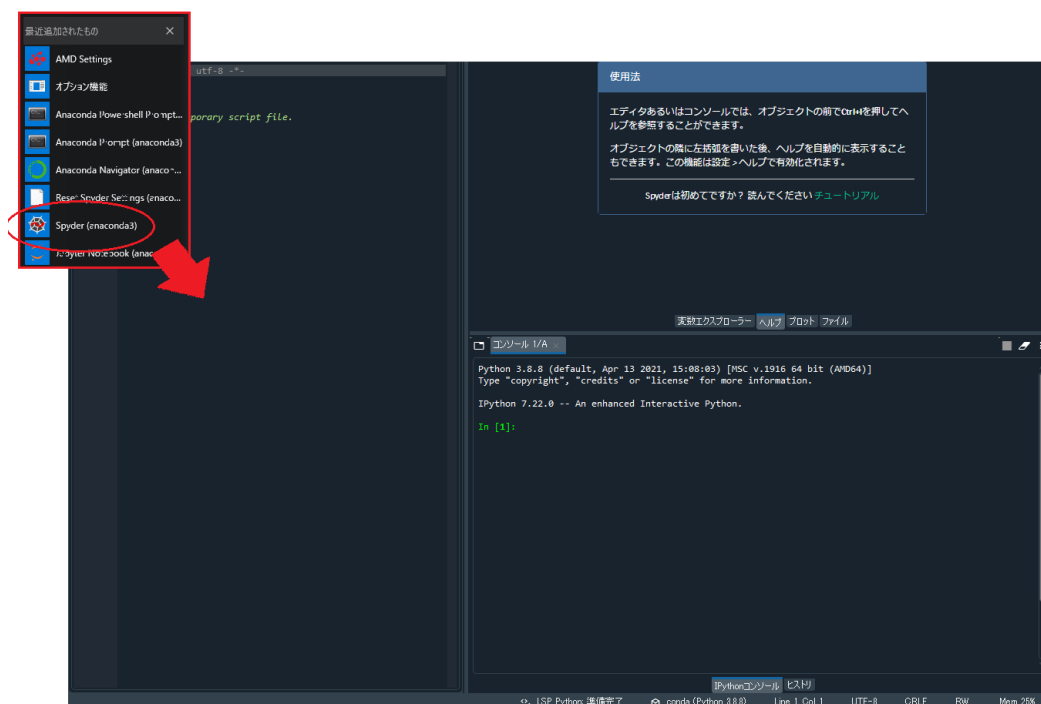
<https://www.anaconda.com/products/individual>



Spyder の起動

エディタ・実行環境の **Spyder** で開発を行います。

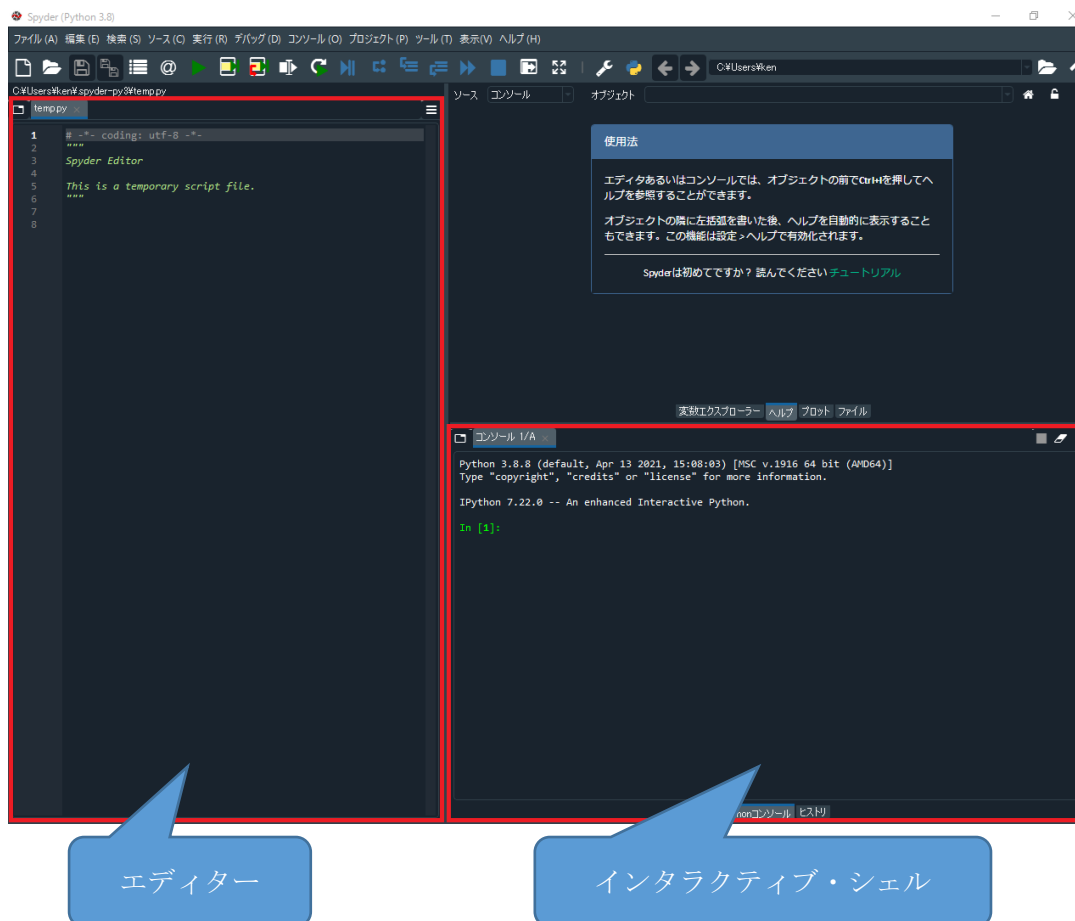
スタートメニューから「すべてのアプリ」→「Anaconda3(64-bit)」→「Spyder」を起動します。



Python プログラムの実行

Python プログラムの実行方法

Python プログラムの実行には、「インタラクティブシェルを用いてダイレクトに実行する方法」と「.py ファイルに記述して実行する方法」の2通りがあります。



長いプログラムを実行する場合は、エディターで記述してから「.py ファイル」として保存し、実行します。

直接命令を入力し、実行するのならインタラクティブ・シェルを用います。

インタラクティブ・シェルを試してみる

次に、インタラクティブ・シェルを使って簡単なプログラムを実行します。



以下の処理を入力してエンターキーを押してみましょう。

```
print("Hello World")
```

すると、「Hello World」と表示されます。

`print()`関数は()内に入れたデータ（文字列や数値など）を出力する関数です。

文字列は「"（ダブルクォーテーション）」または「'（シングルクォーテーション）」で囲みます。

■ 複数出力

```
print(10,20)
```

■ 改行なし

```
print(10,end="") #最後に改行文字を入れない
```

数値演算子

数値計算は、数値演算子を使って行います。**数値演算子**は、次のような四則演算の他に、余りや商の整数値を求めるなどの演算子があります。

演算子	使用例	説明
+	<code>a + b</code>	足し算
-	<code>a - b</code>	引き算
*	<code>a * b</code>	掛け算
/	<code>a / b</code>	割り算
//	<code>a // b</code>	<code>a</code> を <code>b</code> で割った商の整数値（小数点以下を切り捨て）
%	<code>a % b</code>	<code>a</code> を <code>b</code> で割って、割り切れなかった余り（剰余）
**	<code>a ** b</code>	<code>a</code> を <code>b</code> 回かけた値（累乗）

以下のような処理を実行すると、四則演算を行うことができます。

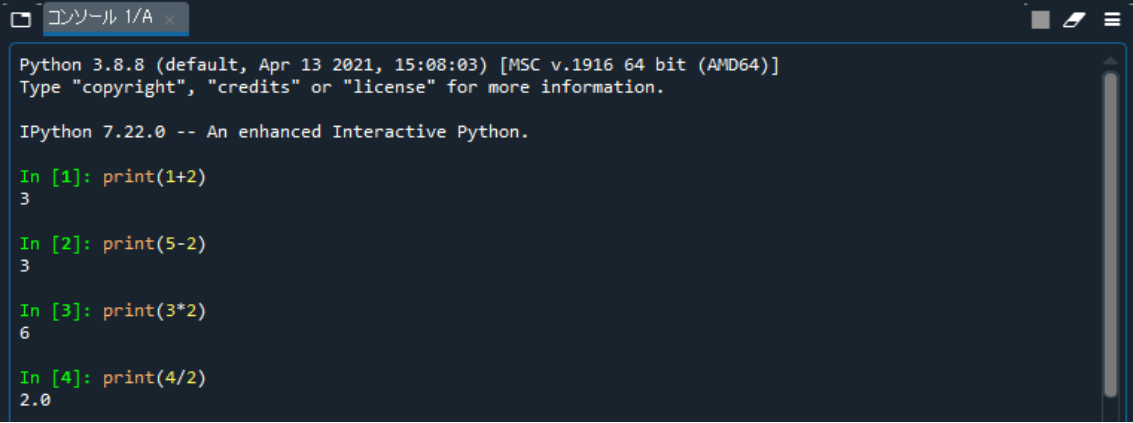
```
print(1 + 1)

print(5 - 2)

print(3 * 2)

print(4 / 2)
```

【実行結果】は以下のようになります。



```
Python 3.8.8 (default, Apr 13 2021, 15:08:03) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.22.0 -- An enhanced Interactive Python.

In [1]: print(1+2)
3

In [2]: print(5-2)
3

In [3]: print(3*2)
6

In [4]: print(4/2)
2.0
```

小数点以下の数値を四捨五入するには `round()` を使います。

■ `round()` の使用例 1

```
print(round(4/2))
```

【実行結果】

```
2
```

■ `round()` の使用例 2

```
print(round(10/4))
```

【実行結果】

```
2
```

端数がちょうど **0.5** の場合は、切り捨てと切り上げのうち、結果が偶数となる方になります。

2 章 変数

変数とは

変数は、データを一時的に入れる箱のようなもので、数値・文字列などが入ります。変数には、1つ1つに名前を付けます。

■ 変数の例

```
num  
message
```

変数に値を入れることを**代入**（だいにゆう）と言います。変数の値は何度でも書き換えることが出来ます。

変えられない値のことは**定数**と言います。Python には定数はありませんが、慣習として大文字で書いた変数を定数として使います。

■ 変数に値を代入する

```
num = 100  
message = "Hello"
```

変数名のルール

変数名には以下のルールがあります。

- 1 文字目は英文字かアンダーバー(_)
- 2 文字目以降は英数文字、アンダーバー
- 予約語は使用できない
- 大文字と小文字は区別される

※ひらがなや漢字などの文字も使えますが、一般的に使用しません。

■ 使ってよい変数の例

```
number    CountryName    bg_color    city20    _num5
```

■ 変数名として使えない名前の例

```
100names    2Baa
```

■ Python の予約語

```
False, None, True, and, as, assert, break, class, continue, def, del, elif,
else, except, finally, for, from, global, if, import, in, is, lambda,
nonlocal, not, or, pass, raise, return, try, while, with, yield
```

■ Python の予約語を確認する例

```
import keyword
print(keyword.kwlist)
```

文字列 (str)

Python 言語の文字列には様々な記述方法があります。「'～」や「"～」のように、シングルクォート「'」、ダブルクォート「"」で囲みます。複数行の文字列は、「'''～」または「"""～」のように三重引用符で囲みます。

■ 文字列の例

```
print('What is New') #シングルクォート 「#」はコメント詳細は後述しています
print("What is New") #ダブルクォート
```

【実行結果】

```
What is New
What is New
```

■ 「'」シングルクォートを文字列として表示

```
print('What's New') #文字列の範囲が不明瞭のためエラー
print("What's New") #文字列の記号がダブルクォートのため正常に実行
```

【実行結果】 ※別々に実行してください

```
print('What's New')はエラーとなる
What's New
```

■ エスケープシーケンス

特別な文字列としてエスケープシーケンスがあります。「¥」と特別な文字の組み合わせで特別な意味を表します。

```
print('What¥'s New')      # 「¥'」で「'」を一つの文字として表示
print('牛井は¥¥300 円')   # 「¥¥」で「¥」を一つの文字として表示
```

【実行結果】

```
What's New
牛井は¥300 円
```

■ raw 文字列の例

raw 文字列は、「¥」をエスケープシーケンスではなく 1 つの文字として扱うことができ、ファイルパスの指定の際などに便利です。

```
print(r'牛井は¥300 円')  
print(r"C:¥Users¥hoge¥Desktop")
```

【実行結果】

```
牛井は¥300 円  
C:¥Users¥hoge¥Desktop
```

■ 三重引用符を使用した文字列の例

```
print("""ABC  
あいうえお  
カキクケコ""")
```

※インタラクティブシェル内での改行は「Ctrl + Enter」で行います。

【実行結果】

```
ABC  
あいうえお  
カキクケコ
```

文字列結合

文字列結合とは言葉の意味通り複数の文字列を繋げて一つの文字列データにする仕組みのことです。「+」を使用して文字列を結合します。

```
name = "菅野"  
hobby = "プロレス観戦"  
print("私の名前は" + name + "です、趣味は" + hobby + "です")
```

【実行結果】

```
私の名前は菅野です、趣味はプロレス観戦です
```

f 文字列

文字列結合を簡易的に記述するために python では f 文字列があります。

f 文字列は、文字列の一部に { } を使用し、その中に変数や式をいれておき、それを実行時に置き換えるというものです。

先ほどのサンプルコードを下記に修正してください。

```
print(f"私の名前は{ name }です、趣味は{ hobby }です")
```

※実行結果は同じため割愛します

[]で文字列を取り出す

文字列はそれぞれの文字の位置をインデックス番号で表します。インデックス番号は、1文字目を0で表し、最後の文字を-1で表します。

0	1	2	3	4	5	6	7	8
a	b	c	d	e	f	g	h	i
-9	-8	-7	-6	-5	-4	-3	-2	-1

■ インデックス番号で文字列を取り出す例

```
word = "abcdefghi"
print(word[2])
print(word[-1])
```

【実行結果】

```
c
i
```

■ 不正なインデックス番号で文字列を取り出すとエラーになる例

```
word = "abcdefghi"
print(word[10])
```

【実行結果】(エラー)

```
Traceback (most recent call last):
  File "<ipython-input-2-1f07c0250e19>", line 2, in <module>
    print(word[10])
IndexError: string index out of range
```

■ あるインデックス番号の位置の文字を変更するとエラーになる例

```
word = "text"
word[2] = "s"
```

【実行結果】(エラー)

```
Traceback (most recent call last):
  File "<ipython-input-1-9e5d3f4ee5be>", line 2, in <module>
    word[2] = "s"
TypeError: 'str' object does not support item assignment
```

Python の文字列 (str) はイミュータブルなオブジェクトなのでエラーになります。
イミュータブルは変更不可、ミュータブルは変更可という意味です。

文字列のスライス

文字列のスライスは、部分文字列を取り出すことです。

■ 書式

変数名[開始インデックス : 終了インデックス]

開始に指定されたインデックス番号の位置から、終了に指定されたインデックス番号の位置の手前までを取り出します。

文字列のスライスは、不正なインデックス番号でもエラーにはなりません。

■ 部分文字列を取り出す（文字列のスライス）例

```
message = "hello world"
print(message[ : ])    #全て
print(message[6 : ])  #7 文字目から最後まで
print(message[6 : 6 + 2]) #7 文字目から 2 文字
print(message[ : 5])   #先頭から 5 文字目まで
print(message[ : -6])  #先頭から、後ろから 6 文字目の手前まで
```

【実行結果】

```
hello world
world
wo
hello
hello
```

0	1	2	3	4	5	6	7	8	9	10
h	e	l	l	o		w	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

コメント

Python 言語のコメントは、シャープ「#」の後ろ記述します。行の先頭に#があれば、その行全体がコメント文になり、行の途中に#があれば、その#の後ろがコメント文です。

複数行のコメントは、三重引用符を使用した文字列と同じように記述します。これは、Python 言語が何も出力しない文字列があってもエラーにならないという仕様を利用しています。

■ コメント記述の例

```
#コメント
```

```
"""
```

```
複数行コメント 1
```

```
複数行コメント 2
```

```
複数行コメント 3
```

```
"""
```

整数型（int）と浮動小数型（float）

Python 言語の整数型は 10 進数、2 進数、8 進数、16 進数が表現できます。浮動小数型は、一般的に 8 バイトの浮動小数です。

■ 整数型（2 進数）の例

```
print(0b1010)
```

【実行結果】

```
10
```

結果は 10 進数で表示されます。

■ 整数型（8 進数）の例

```
print(0o12)
```

【実行結果】

```
10
```

■ 整数型（16 進数）の例

```
print(0x12)
```

【実行結果】

```
18
```

■ 浮動小数型の例

```
print(10/4)
print(10/5)
```

【実行結果】

```
2.5
2.0
```

整数同士の割り算は、値が割り切れても割り切れなくても結果は浮動小数型になります。

ブール型 (bool)

bool 型は True(真) と False(偽) でデータを表すデータのタイプの一種です。条件の真偽を表すために用いる特殊な型です。数値や文字列などと並び大事なデータ型です。

どちらも先頭の文字が大文字なのです。間違えないように注意してください。

Python 言語では、数値の 0 は False、その他の数値（一般的に 1）は True と同じ値として使われます。

累算代入演算子

累算代入演算子は、数値が代入された変数に対して使用すると、数値演算した値を代入することができる演算子です。

演算子	使用例	説明
+=	a += b	a に b を足して、a に代入
-=	a -= b	a から b を引いて、a に代入
*=	a *= b	a に b を掛けて、a に代入
/=	a /= b	a を b で割って、a に代入
//=	a //= b	a を b で割った商の整数値を a に代入
%=	a %= b	a を b で割って、割り切れなかった余りを a に代入
**=	a **= b	a を b 回かけた値を a に代入

■ 累算代入演算の例

```
num = 10
num *= 2
print(num)
```

【実行結果】

```
20
```

num に 10 を代入し、それを 2 倍にすることから、num は 20 になります。

演算子の優先順位

乗算・除算は、加算・減算よりも優先します。数学のルールと同じで()の計算は優先されます。

■ 演算子の優先順位の例 1

```
print( (4 + 2) / 2 )
```

【実行結果】

```
3.0
```

■ 演算子の優先順位の例 2

```
print( 4 + 2 / 2 )
```

【実行結果】

```
5.0
```

このように()を付けた場合と付けない場合で演算の結果が違います。

型変換

数値を文字列に変換する場合は **str()** を使用します。

数値や文字列を整数に変換する場合は **int()** を使用し、浮動小数に変換する場合は **float()** を使用します。

■ str()の例

```
area = 5 * 5 * 3.14
result = "円の面積は" + str(area) + "m2です"
print(result)
```

【実行結果】

```
円の面積は 78.5 m2です
```

文字列結合は「+」を使用します。ただし、文字列同士しか結合できないので、数値は **str()** で変換してから結合します。

■ int()の例

```
price = int(2800 * 1.1)
result = "税込み" + str(price) + "円です"
print(result)
```

【実行結果】

```
税込み 3080 円です
```

■ float()の例

```
area = 3 * 3 * float("3.14")
result = "円の面積は" + str(area) + " m2です"
print(result)
```

【実行結果】

```
円の面積は 28.26 m2です
```

型を調べる

値や変数の型は `type()` で調べることができます。

■ type()の例

```
price = int(2800 * 1.1)
result = "税込み" + str(price) + "円です"
print(type(price))
print(type(result))
```

【実行結果】

```
<class 'int'>
<class 'str'>
```

3 章 条件分岐

if 文

プログラムにおける「分かれ道」を**条件分岐**と言います。

Python 言語では、条件分岐の処理に「**if 文**」を用います。**if** は「もしも」という意味で、ある条件が成り立つ場合にだけ処理を行います。

■ if 文の書式

```
if 条件 :
```

```
    処理 A
```

```
処理 B
```

※処理 A にはインデント（TAB キー）が必要です

条件が成り立つ場合、「 処理 A → 処理 B 」の順で実行されます。

処理 A に当たる部分が複数行ある場合は、各行に**インデント**が必要です。

条件が成り立たない場合、「 処理 B 」のみ実行されます。

■ if 文の例

```
age = 28
```

```
if age >= 20:
```

```
    print('20 歳以上です')
```

```
print('年齢確認終了です')
```

【実行結果】

```
20 歳以上です
```

```
年齢確認終了です
```

比較演算子

if 文の条件式では以下の比較演算子を用います。

記号	使用例	意味
<	a < 5	より小さい
<=	a <= -5	以下
>	b > 22	より大きい
>=	b >= -3.1	以上
==	i == 4	等しい
!=	num != -1	等しくない

条件式は **bool 型** で値を返す処理です。命題の内容が正しければ **True** を、間違っていれば **False** を返します。

■ 比較演算子の例 1

```
print(6 > 3)
```

【実行結果】

```
True
```

■ 比較演算子の例 2

```
print(-13 == 12)
```

【実行結果】

```
False
```

if～else 文

if 文だけでは表現しきれない複雑な条件分岐を紹介します。

例えば、「晴れたら運動会、雨なら授業」といったように、ある条件が成り立つ場合と成り立たない場合で行う処理が異なる場合です。

else (エルス) 文は、if 文の条件が成り立たなかった場合に実行される処理を記述できます。

■ if～else 文の書式

```
if 条件 :
```

```
    条件が True の時の処理
```

```
else:
```

```
    条件が False の時の処理
```

■ if～else の例

```
age = 18
```

```
if age >= 20:
```

```
    print('20 歳以上です')
```

```
else:
```

```
    print('20 歳未満です')
```

【実行結果】

```
20 歳未満です
```

if～elif～else 文

条件が複数ある場合は、**elif** 文を用います。

例えば、「信号の色が青は進め、黄また赤は止まれ、黄点滅は注意して進め」といったような条件が複数ある場合です。

書式は「**if ～ elif ～ else**」のようになります。**if** 文が成り立たない場合も複数の条件式を提示できます。

■ if～elif～else 文の書式

```
if 条件 1 :  
    条件 1 が True の時の処理  
elif 条件 2 :  
    条件 1 が False で条件 2 が True の時の処理  
elif 条件 3 :  
    条件 1・条件 2 が共に False で条件 3 が True の時の処理  
else:  
    全ての条件が False の時に実行する処理
```

■ if～elif～else の例

```
a = 2  
if a == 1:  
    print("a==1")  
elif a == 2:  
    print("a==2")  
else:  
    print("a は 1、2 以外の数")
```

【実行結果】

```
a==2
```

論理演算子

条件分岐の条件が一つであるとは限りません。条件式では複数の条件式を扱うことができます。複数の条件が全て成り立つ条件は **and**、複数の条件の何れかが成り立つ条件は **or** を用います。

論理積（and）

複数の条件が全て成り立つ（True）とき、真（True）になる条件を論理積と言います。
論理積は **and** で表現します。

条件 A	条件 B	条件 A and 条件 B
True	True	True
True	False	False
False	True	False
False	False	False

■ 論理積による条件の書式

```
if 条件 A and 条件 B :  
    条件 A と条件 B が共に True の時の処理
```

■ 論理積による条件の例

```
m = 1  
n = 1  
if m == 1 and n == 1:  
    print("m も n も 1 です。")
```

【実行結果】

```
m も n も 1 です。
```

論理和（OR）

複数の条件の何れかが成り立つ（True）のとき、真（True）になる条件を論理和と言います。

条件 A	条件 B	条件 A or 条件 B
True	True	True
True	False	True
False	True	True
False	False	False

■ 論理和による条件の書式

```
if 条件 A or 条件 B :  
    条件 A と条件 B の何れかが True の時の処理
```

■ 論理和による条件の例

```
m = 1
n = 1
if m == 1 or n == 1:
    print("m か n が 1 です。")
```

【実行結果】

```
m か n が 1 です。
```

否定 (not)

論理演算子には `and` と `or` 以外に **not** があります。「`not a == b`」のように使用して、「`a == b`」が `True` なら `False` になり、「`a == b`」が `False` なら `True` になります。つまり、`True` と `False` を反転することができます。

■ 否定による条件の例

```
m = 5
if not(m == 1):
    print("m は 1 ではありません。")
```

【実行結果】

```
m は 1 ではありません。
```

if 文のネスト

`if` 文の中に、更に `if` 文が記述することを **if 文のネスト** と言います。`elif` 文や `else` 文でも `if` 文のネストは可能です。

■ if 文のネストの書式

```
if 条件 1 :
    if 条件 2 :
        条件 A と条件 B が共に True の時の処理
```

■ if 文のネストの例

```
age = 20
if age >= 20:
    if age < 60:
        print("20 歳以上 60 歳未満")
```

【実行結果】

```
20 歳以上 60 歳未満
```

4 章 繰り返し処理

while 文

指定した回数、同じ処理を繰り返すことを**繰り返し処理（ループ処理）**と言います。

Python 言語では、**while 文**を用いて実装します。条件が成り立っている間、処理を繰り返します。

■ while 文の書式

```
while 条件 :  
    繰り返す処理
```

while 文の条件は if 文の書式と同じです。if 文との違いは、条件が成り立っている間、処理を繰り返します。繰り返し処理の部分が複数行ある場合は各行にインデントが必要です。

■ while 文の例 1

```
i = 0  
while i < 3:  
    print(i)  
    i = i + 1      # i += 1
```

【実行結果】

```
0  
1  
2
```

この処理では初期値として **i** に **0** を代入し、**i** が **3** 未満の間、**while** 以下の処理を繰り返します。**i = i + 1** により毎回 **i** の値が **1** 増えていくので、**i = 3** の時に **i < 3** が **False** となるので繰り返し処理が終了します。

■ while 文の例 2

```
i = 5  
while i > 0:  
    print(i)  
    i = i - 1      # i -= 1
```

【実行結果】

```
5  
4  
3  
2  
1
```

この処理では初期値として **i** に **5** を代入し、**i** が **0** より大きい間、**while** 以下の処理を繰り返します。**i = i - 1**により毎回 **i** の値が **1** 減っていくので、**i = 0** の時に **i > 0** が **False** となるので繰り返し処理が終了します。

無限ループ

処理が終わらない繰り返しの無限ループと言います。プログラムを強制終了しない限り止まりません。

■ 無限ループの例

```
while True:
    print("hoge")
```

【実行結果】

```
hoge
hoge
hoge
...
```

while 文の条件が常に **True** なら、無限ループになります。

強制終了するには「**Ctrl + C**」を入力します。強制終了しない限り終わりません。

break 文

ループ処理の中で **break** 文が実行されると、ループ処理から抜け出せます。**break** 文が実行される条件は **if** 文で指定します。

■ break 文の例

```
i = 0
while i < 5:
    i += 1
    if i == 3:
        break
    print(i)
```

【実行結果】

```
1
2
```

i が **3** のときに **break** 文を実行され、強制的にループから抜け出せます。

continue 文

ループ処理の中で **continue** 文が実行されると、ループ処理の先頭にジャンプすることができます。continue 文が実行される条件も **break** と同じく **if** 文で指定します。

■ continue 文の例

```
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue
    print(i)
```

【実行結果】

```
1
3
5
7
9
```

i が偶数(**i** を 2 で割った余りが **0**)のときに **continue** 文が実行され、強制的にループの先頭にジャンプするため、【実行結果】は奇数だけが表示されます。

多重ループ（繰り返しのネスト）

ループ処理の中にループ処理が入っている処理を**多重ループ**と言います。ループ処理が 2 つなら二重ループ、3 つなら三重ループと言います。

■ 多重ループの例

```
i = 0
while i < 2:
    j = 0
    while j < 3:
        print("i:", i, "j:", j)
        j += 1
    i += 1
```

【実行結果】

```
i: 0 j: 0
i: 0 j: 1
i: 0 j: 2
i: 1 j: 0
i: 1 j: 1
i: 1 j: 2
```

外側のループでは、**i** を **0** から **1** ずつ増やしていき、**0**、**1** と増加していきます。内側のループでも **j** が **0**、**1**、**2** と変化していきます。

`print("i:", i, "j:", j)` で、**i** と **j** の組み合わせが表示されます。内側で 3 回、外側で 2 回のループが繰り返されるため、6 回（ 3×2 ）処理が実行されます。

繰り返し処理（ループ処理）は、**while** 文の他に **for** 文でも記述することができます。**for** 文はこの後紹介する**データ構造**と密接な関係にあるため、データ構造と共に説明します。

5 章 データ構造

データ構造とは何か

コンピューターがデータを効率的に扱うために決めた形式を**データ構造**と言います。大量のデータや複雑なデータを扱わなくてはならない場合にデータ構造を用います。Python 言語で扱えるデータ構造には、以下のものがあります。

- リスト (**list**)
- タプル (**tuple**)
- 集合 (**set**)
- 辞書 (**dict**)

【データ構造特徴】

	検索	追加	更新	削除	データ管理
リスト	●	●	●	●	インデックス
タプル	●				インデックス
集合	●	●	●	●	なし
辞書	●	●	●	●	キー

リスト (**list**)

複数のデータを同時に扱える変数をリスト (**list**) と言います。**インデックス (添え字)** と呼ばれる数値で管理し、データの追加・削減が容易にできます。プログラムの中で大量のデータを扱うときに便利です。データ構造の中では一番使用頻度の高いものです。

■ リスト (**list**) の書式

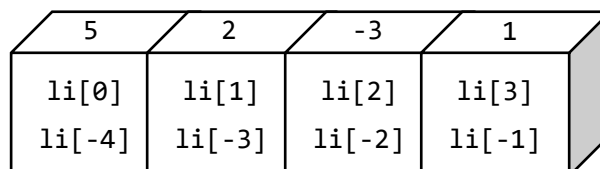
リスト名 = [値 1, 値 2, 値 3,]

例) `li = [5, 2, -3, 1]`

リストは 1 つ 1 つの要素が独立した変数として扱えることから、次のように `li[0]` ~ `li[3]` の 4 つの変数 (要素) として扱うことができます。各要素は後から値を変えることもできます。

■ リスト **li** の各要素の値

```
li[0] ... 5
li[1] ... 2
li[2] ... -3
li[3] ... 1
```



リストへのデータの追加 : `append()`メソッド

`append()`メソッドは、リストの末尾にデータを追加することができます。

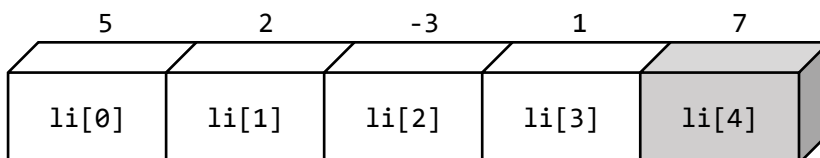
■ `append()`メソッドの使用例

```
li = [5 , 2 , -3 , 1]
li.append(7)
print(li)
```

【実行結果】

```
[5, 2, -3, 1, 7]
```

「`li.append(7)`」で末尾に新しい要素 `li[4]`が追加され、値は7になります。



リストへのデータの挿入 : `insert()`メソッド

`insert()`メソッドは、リストの指定したインデックスにデータを挿入することができます。

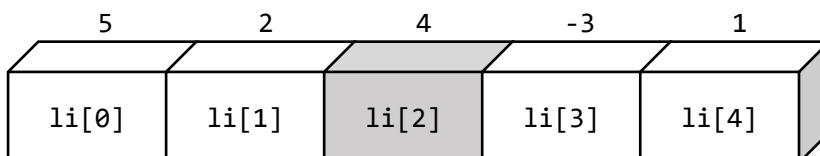
■ `insert()`メソッドの使用例

```
li = [5 , 2 , -3 , 1]
li.insert(2, 4)
print(li)
```

【実行結果】

```
[5, 2, 4, -3, 1]
```

「`li.insert(2, 4)`」で `li[2]`に新しい要素が追加され、値は4になります。



リストからのデータの削除：remove()メソッド

remove()メソッドは、リストから指定したデータを削除することができます。しかし、リスト内に指定したデータがない場合、エラーが発生します。

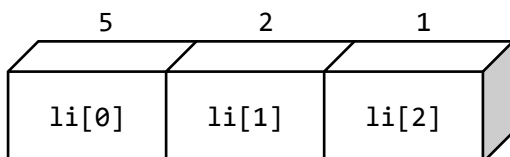
■ remove()メソッドの使用例

```
li = [5 , 2 , -3 , 1]
li.remove(-3)
print(li)
```

【実行結果】

```
[5, 2, 1]
```

「**li.remove(-3)**」で **li[2]** の値が **-3** ですので、この要素が削除されます。もし、同じ値を持つ要素が他にもある場合は、インデックスが若いものが削除されます。



■ remove()メソッドのエラー対策の例

```
val = 10
li = [5 , 2 , -3 , 1]
if val in li:
    li.remove(val)
print(li)
```

if 文の条件「**val in li**」は、リスト **li** の中に **val** が見つかったら **True**、見つからないと **False** となります。ですので、リスト **li** の中に **val** が見つかった場合のみ削除されます。

※**in** では **int** など 1 つのデータ型には使用できず、インデックスがあるものやデータ構造型に使用可能です。

リストからのデータの削除：pop()メソッド

remove()メソッド以外に **pop()**メソッドも、リストから指定したインデックスのデータを抜き取る（削除する）ことができます。しかし、リスト内に指定したデータがない場合、エラーが発生します。

■ pop()メソッドの使用例 1

```
li = [5 , 2 , -3 , 1]
li.pop(1)
print(li)
```

【実行結果】

```
[5 , -3 , 1]
```

「li.pop(1)」で li[1]の要素が削除されます。

■ pop()メソッドの使用例 2

```
li = [5 , 2 , -3 , 1]
li.pop()
print(li)
```

【実行結果】

```
[5 , 2 , -3]
```

「li.pop()」で末尾の要素（li[3]の要素）が削除されます。

■ pop()メソッドのエラー対策の例

```
li = [5 , 2 , -3 , 1]
length = len(li)
index = 10
if index >=0 and index < length:
    li.pop(index)
print(li)
```

【実行結果】

```
[5, 2, -3, 1]
```

「len()関数」を使用すると、リストのサイズ（要素数）を取得できます。ですので、リスト li の正当なインデックスが指定された場合のみ削除されます。

リストのスライス

■ 書式

リスト名[開始インデックス: 終了インデックス]

文字列のスライスと同じようにスライスが可能です。

■ スライスの使用例

```
li = [5 , 2 , -3 , 1]
print(li[1:3])
```

【実行結果】

```
[2, -3]
```

リストのデータのクリア: clear()メソッド

clear()メソッドは、リストから全データを削除することができます。

■ clear()メソッドの使用例

```
li = [5 , 2 , -3 , 1]
li.clear()
print(li)
```

【実行結果】

```
[]
```

[]は、空のリスト（リストの中に何もデータが入っていない状態）を意味します。
空のリストを作成する時には「**変数 = []**」のような書式になります。

for 文によるリストの繰り返し処理

for 文を用いるとリストの要素数だけ繰り返し処理を記述することができます。繰り返し処理の中でリストの各要素内にアクセスすることができます。

■ for 文の書式

```
for 変数 in リスト :  
    要素の数だけ繰り返すコード
```

■ for 文の例

```
names = ['山田', '佐藤', '工藤', '大山']  
for name in names:  
    print(name)
```

【実行結果】

```
山田  
佐藤  
工藤  
大山
```

このように、リストの各要素内にアクセスすることができます。

for 文のその他の使い方

for 文は、リスト以外にも **range()関数**とセットでよく用いられます。**range()**は指定した範囲の整数の領域を表す関数です。**range()**関数を使用すると **range** 型が作られますが、**list** 型と違い、参照はできますが追加や更新はできません。

■ range()関数の書式

```
range(5) → 0, 1, 2, 3, 4  
range(2, 5) → 2, 3, 4
```

for 文は **range()**関数を利用して、指定した範囲内の数値の処理を行うことができます。

■ for 文で range()関数を利用した例

```
for num in range(1, 3):  
    print(num)
```

【実行結果】

```
1  
2
```

range()関数を利用した連番リストの作成

range()関数を利用して指定した範囲内の数値を持つデータを、list()関数で作成することができます。

■ 1～10の連番リストを作成した例

```
li = list(range(1, 11))  
print(li)
```

【実行結果】

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

リスト (list) の内包表記

リスト内包表記は、[]の中に for-in の構文を記述して新しいリストを作成できます。

■ リスト内包表記の書式

```
[ 式 for 変数 in オブジェクト ]
```

オブジェクトには、リスト、文字列、range()関数などが指定できます。式には新しく作成したリストに登録する値を指定します。

■ リスト内包表記の例1

```
li = [5, 2, -3, 1]  
li_double = [num * 2 for num in li]  
print(li_double)
```

【実行結果】

```
[10, 4, -6, 2]
```

■ 内包表記を使用しなかった場合

```
li = [5, 2, -3, 1]  
li_double = []  
for num in li:  
    li_double.append(num * 2)  
print(li_double)
```

※実行結果は同じため割愛します。

■ リスト内包表記の例 2

```
classes = [cl + 'クラス' for cl in 'ABCDE']  
print(classes)
```

【実行結果】

```
['A クラス', 'B クラス', 'C クラス', 'D クラス', 'E クラス']
```

■ 条件付きのリスト内包表記の書式

```
[ 式 for 変数 in リスト if 条件式 ]
```

指定したリストの中から、条件が成り立つ要素だけの新しいリストを作成できます。

■ 条件付きのリスト内包表記の例

```
li = [5 , 2 , -3 , 1, -2, 7, 10]  
li_even = [num for num in li if num % 2 == 0]  
print(li_even)
```

【実行結果】

```
[2, -2, 10]
```

■ 内包表記を使用しなかった場合

```
li = [5, 2, -3, 1, -2, 7, 10]  
li_even = []  
for num in li:  
    if num % 2 == 0:  
        li_even.append(num)  
print(li_even)
```

※実行結果は同じため割愛します。

タプル (tuple)

タプル (tuple) とはリストに似ていますが、**要素の値は変更できません** (イミュータブル)。また、追加も削除もできませんが、リストに比べて処理速度は速く、メモリも消費しないのが特徴です。内包表記はありません。

■ タプル (tuple) の書式

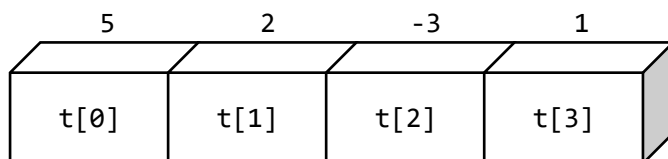
タプル名 = (値 1, 値 2, 値 3 . . .)

例) t = (5 , 2 , -3 , 1)

「()」を省略することもできます。

■ タプル t の各要素の値

```
t[0] ... 5
t[1] ... 2
t[2] ... -3
t[3] ... 1
```



タプル (tuple) はリスト (list) と同様、要素にアクセス (取得) することができますが、値を変更することはできません。

■ タプル (tuple) の使用例

```
t = ( 5 , 2 , -3 , 1 )
print(t[1])
t[1] = 5
```

【実行結果】

```
2
... エラーメッセージは省略
```

「t[1] = 5」はエラーになります。タプルは値を変更できないからです。

■ タプル (tuple) とリスト (list) の相互変換の例

タプルは値の追加や更新ができないため、データを変更したい場合は一度 **list** に変換する必要があります。また、**list** のデータを高速化するためタプルに変換することがあります。

```
t1 = (5 , 10 , 15)
li1= list(t1)
li2 = [10, 20, 30]
t2 = tuple(li2)
print(li1)
print(t2)
```

【実行結果】

```
[5, 10, 15]
(10, 20, 30)
```

li1 にはタプルをもとに作成したリストが代入され、**t2** にはリストをもとに作成したタプルが代入されます。

タプルのアンパック

変数の代入は「変数 = 値」のように、左辺に変数、右辺に値を記述します。同様に「(a, b) = (10, 20)」のように、左辺にタプルの書式で変数を指定して、右辺にタプルの書式で値を指定すると、**a** に **10** が代入され、**b** に **20** が代入されます。

これを「タプルのアンパック」と言います。アンパックの使用により、タプルの要素を展開し複数の変数に代入することが出来ます。

■ タプルのアンパックの使用例

```
(x, y, z) = (1 , 10 , 100)
a, b = x + y, y * z
print(x)
print(y)
print(z)
print(a)
print(b)
```

※【実行結果】は次のページ

【実行結果】

```
1
10
100
11
1000
```

「`(x, y, z) = (1 , 10 , 100)`」のように、左辺には右辺のタプルの要素の数と同じ数の変数、右辺には左辺の変数と同じ数の要素をもつタプルを記述します。また、左辺と右辺ともに「`()`」は省略できます。

「`a, b = x + y, y * z`」のように記述すると、左辺の変数に、右辺の計算結果などを代入することもできます。

※リストを使用したアンパックも可能ですが、`[]`は省略できません。

集合 (set)

集合 (set) とはリストに似ていますが、同じ値のデータを持つことができません。また、集合演算 (しゅうごうえんざん) を行うことが可能です。

■ 集合 (set) の書式

集合名 = {値 1, 値 2, 値 3 . . . }

例) `s = {1, 6, -3, 2}`

集合 (set) は「{ }」の間に要素を指定します。

■ 集合 (set) の使用例

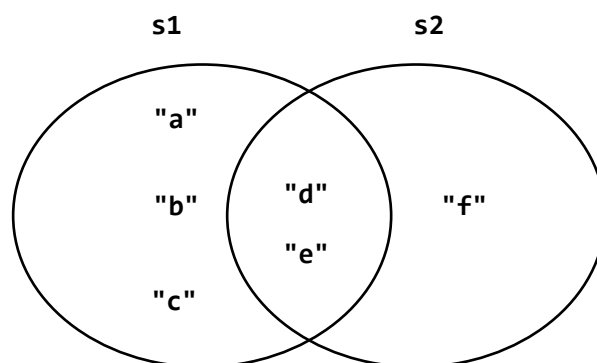
```
people = {'Tom', 'Mike', 'Bob'}
number = {1, 2, 2, 3, 3, 4}
print(people)
print(number)
```

【実行結果】

```
{'Bob', 'Tom', 'Mike'}
{1, 2, 3, 4}
```

集合は重複した値を持ちません。定義時に重複があっても無視されます。このように、集合に入る値は必ず重複しないようになっています。

また、追加: `add()`、削除: `remove()`、全削除 `clear()` といったデータ操作が可能です。



集合 (set) は、集合演算を行うことが可能です。

- 和集合 ($s1 \mid s2$) : $s1$ と $s2$ の要素をあわせる
- 積集合 ($s1 \& s2$) : $s1$ と $s2$ の共通要素のみにする
- 差集合 ($s1 - s2$) : $s1$ から $s2$ の要素を取り除く
- 対象差集合 ($s1 \wedge s2$) : $s1$ と $s2$ の共通要素を取り除く

■ 集合演算の例

```
s1 = {'a', 'b', 'c', 'd', 'e'}
s2 = {'d', 'e', 'f'}
print("和集合", s1 | s2)
print("積集合", s1 & s2)
print("差集合", s1 - s2)
print("対象差集合", s1 ^ s2)
```

【実行結果】

```
和集合 {'f', 'a', 'c', 'b', 'd', 'e'}
積集合 {'d', 'e'}
差集合 {'a', 'c', 'b'}
対象差集合 {'a', 'c', 'b', 'f'}
```

セット（Set）の内包表記

セット内包表記は、`{}`の中に `for-in` の構文を記述して新しいセットを作成できます。セット内包表記もリスト内包表記と同じような書式です。

■ セット内包表記の書式

```
{ 式 for 変数 in オブジェクト }
```

オブジェクトには、リスト、文字列、`range()`関数などが指定できます。

■ セット内包表記の例

```
li = [5 , 2 , -3 , 1 , 4 , 2 , 1]
set_double = {num * 2 for num in li}
print(set_double)
```

【実行結果】

```
{2, 4, 8, 10, -6}
```

■ セット内包表記の例

```
li = [5 , 2 , -3 , 1 , 4 , 2 , 1]
set_even = {num for num in li if num % 2 == 0}
print(set_even)
```

【実行結果】

```
{2, 4}
```

辞書 (dict)

辞書 (dict) とは、各要素をキー (key) と値 (value) の組み合わせで管理する方法です。辞書のようにキーをもとに検索して値を取得します。

辞書の実装は、以下のように行います。

■ 辞書の書式

```
辞書名 = {キー名 1:値 1, キー名 2:値 2, キー名 3:値 3・・・}  
例) d = {'yellow' : '黄色', 'red' : '赤', 'blue' : '青'}  
  
辞書名 = dict(キー名 = 値 1)  
例) d2 = dict(id = 1234, name = 'KEN')
```

辞書の作成方法は2通りありますが、dict 関数を使用する場合、キー名は文字列扱いになりますが、引用符 (「'～」や「"～」) で囲うとエラーとなります。また数字をキー名に指定することはできません。

■ 辞書 d の各要素の値

```
d['yellow'] ... '黄色'  
d['red'] ... '赤'  
d['blue'] ... '青'  
d2['id'] ... 1234  
d2['name'] ... 'KEN'
```

リストの添え字に当たるものが、自由に設定できます。キー、値、共に任意のタイプのデータを定義できます。数値・文字列、その他のデータでもかまいません。

辞書への要素追加

辞書に要素を追加するには新たな「キー」と「値」の組み合わせを代入するだけです。

■ 辞書へのデータ追加の例

```
d = {'yellow' : '黄色', 'red' : '赤', 'blue' : '青'}  
d['green'] = '緑'  
d['yellow'] = '黄'  
print(d)
```

【実行結果】

```
{'yellow': '黄', 'red': '赤', 'blue': '青', 'green': '緑'}
```

新しいキーを指定して代入を行うだけで要素が追加されます。また、既存のキーを使って代入した場合は上書きされます。

辞書の要素削除：del 文

辞書の特定の要素を削除するには `del` 文を使用します。全ての要素を削除するには `clear()` メソッドを使用します。

■ 特定の要素を削除する書式

```
del 変数名['キー名']
```

■ 辞書から特定の要素を削除する例

```
d = {'yellow': '黄色', 'red': '赤', 'blue': '青', 'green': '緑'}
del d['red']
print(d)
```

【実行結果】

```
{'yellow': '黄色', 'blue': '青', 'green': '緑'}
```

正当なキーが指定されなかった場合はエラーが発生します。

辞書の複数要素の更新：update()メソッド

複数要素の更新には `update()` メソッドを使用します。ただし、`update()` メソッドには、注意が必要です。既存のキーは更新されますが、存在しないキーは新規登録になります。

`update()` メソッドの引数には、辞書、リスト、タプルを指定することができます。

■ 引数に辞書を指定して、複数要素を更新する例

```
d = {'yellow': '黄色', 'red': '赤'}
d.update({'yellow': '#ff0', 'red': '#f00', 'blue': '#00f'})
print(d)
```

【実行結果】

```
{'yellow': '#ff0', 'red': '#f00', 'blue': '#00f'}
```

既存キーの「yellow」と「red」は更新され、新しいキーの「blue」は新規登録されます。

■ 引数にネストしたリストを指定して、複数要素を更新する例

```
d = {'yellow': '黄色', 'red': '赤'}
d.update(['yellow', '#ff0'], ['red', '#f00'], ['blue', '#00f'])
print(d)
```

【実行結果】

```
{'yellow': '#ff0', 'red': '#f00', 'blue': '#00f'}
```

ネストしたリストは、リストの各要素がリストになっています。

for 文による辞書の繰り返し処理

辞書に for 文を用いるときは以下のような書式になります。

■ for 文による辞書の繰り返し処理の書式

```
for キーの変数, 値の変数 in 辞書.items():  
    要素の数だけ繰り返すコード
```

■ for 文による辞書の繰り返し処理の例

```
d = {'yellow': '黄色', 'red': '赤', 'blue': '青', 'green': '緑'}  
for k, v in d.items():  
    print(k, v)
```

【実行結果】

```
yellow 黄色  
red 赤  
blue 青  
green 緑
```

■ キーの取得

辞書からキー情報のみを必要とする場合は `keys()` を使用します。

```
for k in d.keys():  
    print(k)
```

【実行結果】

```
yellow  
red  
blue  
green
```

■ 値の取得

辞書から値のみを必要とする場合は `values()` を使用します。

```
for v in d.values():  
    print(v)
```

【実行結果】

```
黄色  
赤  
青  
緑
```

辞書 (dict) の内包表記

辞書内包表記は、`{}`の中に `for-in` の構文を記述して新しい辞書を作成できます。

■ 辞書内包表記の書式

```
{ キー: 値 for 変数 in オブジェクト }
```

オブジェクトには、リスト、文字列、`range()`関数などが指定できます。

■ セット内包表記の例

```
colors = ['yellow', 'red', 'blue', 'green']
dict_colors = {color: len(color) for color in colors}
print(dict_colors)
```

【実行結果】

```
{'yellow': 6, 'red': 3, 'blue': 4, 'green': 5}
```

■ 内包表記を使用しなかった場合

```
colors = ['yellow', 'red', 'blue', 'green']
dict_colors = {}
for color in colors:
    dict_colors[color] = len(color)
print(dict_colors)
```

実行結果は同じため割愛します。

6 章 関数

関数とは何か

関数とは複数の処理を一つにまとめて名前を与えたものです。複雑な処理や、何度も繰り返し行うような処理は関数にまとめておくと大変便利です。

Python にはもともと多数の関数が定義されています。

■ Python の関数の例

```
print( 引数 )  
len( 引数 )
```

ユーザーが自分で関数を定義することができるため、ここでは関数を定義し実行する方法について説明します。

関数の定義

関数は以下のように定義します。

■ 関数の定義

```
def 関数名(引数 1, 引数 2, ...):  
    処理 1  
    処理 2  
    ...  
    return 戻り値
```

引数（ひきすう）とは、関数に渡すパラメータで、省略も可能です。引数の値は変数として扱えます。複数ある場合は引数を「，（カンマ）」で区切ります。

戻り値（もどりち）とは、関数が返す処理結果で、こちらも省略することが可能です。計算結果などを返すときに使います。戻り値は「**return 文**」で返します。

関数の呼び出し

以下に関数の実例とその呼び出し方法を紹介します。

■ 引数・戻り値共にある場合の例

```
def add(x, y):  
    return x + y  
  
ans1 = add(2, 3)  
print(ans1)
```

【実行結果】

5

「ans1 = add(2, 3)」のように記述すると、add()関数が呼び出されます。

まず、引数 x に 2 が、引数 y に 3 が代入されます。次に「x + y = 5」が計算され、それを戻り値として返され、ans1 に代入されます。

■ 戻り値がない場合の例

```
def greet(name):  
    print(name + "さん、こんにちは。")  
  
greet("KEN")
```

【実行結果】

KEN さん、こんにちは。

「greet("KEN")」のように記述すると、greet()関数が呼び出されます。

まず、引数 name に "KEN" が代入され、次に「print(name + "さん、こんにちは。")」によって表示されます。この関数は、値を返さないなので return 文は省略されています。

関数を呼び出す場合の注意点

■ 関数は呼び出す前に定義しておくこと

関数を呼び出す際、あらかじめ定義してから呼び出すようにしなくてはなりません。

■ 正しい方法

```
def add(x, y):  
    return x + y  
  
ans1 = add(2, 3)
```

■ 間違った方法

```
ans1 = add(2, 3)  
  
def add(x, y):  
    return x + y
```

■ 関数名の重複の禁止

関数名を重複して定義することはできません。ですので、同一の関数名で異なる引数・戻り値を持つ関数を一度に定義することはできません。

■ 間違った方法

```
def add(x, y):  
    return x + y  
  
def add(x, y, z)  
    return x + y + z
```

可変長引数

Python 言語の関数では、長さ（数）を自由に変えることができる引数を定義することができます。これを**可変長引数**（かへんちょうひきすう）と言います。

与える引数の長さ（数）が一定ではない場合に用います。

可変長引数には、引数が**タプル(tuple)**になるケースと、**辞書(dict)**になるケースの2つが存在します。

■ 引数がタプルになるケース

タプルの可変長引数を持つ関数の定義は以下のようになります。

■ 可変長引数（タプル）を持つ関数定義の書式

```
def 関数名(*引数) :  
    処理 1  
    処理 2  
    ...  
    return 戻り値
```

引数の前に「*」をつけると、引数は**タプル**になります。このように定義することにより、任意の引数を渡すことができます。

■ 可変長引数（タプル）を持つ関数定義と呼び出しの例

```
def show_items(*items):  
    print('「', items[0], '」')  
    i = 1  
    for item in items:  
        print(i, ': ', item)  
        i += 1  
  
show_items('ONE', 'TWO', 'THREE', 'FOUR')  
print('-' * 10)  
show_items(1, 2, 3)
```

【実行結果】

```
「 ONE 」  
1 : ONE  
2 : TWO  
3 : THREE  
4 : FOUR  
-----  
「 1 」  
1 : 1  
2 : 2  
3 : 3
```

`show_items()`関数の引数である `items` は「タプル」です。要素の追加・更新・削除をすることができないので注意してください。

■ 引数が辞書になるケース

辞書の変長引数を持つ関数の定義は以下のようになります。

■ 可変長引数（辞書）を持つ関数定義の書式

```
def 関数名(**引数) :  
    処理 1  
    処理 2  
    ...  
    return 戻り値
```

引数の前に「**」をつけると、引数は辞書になります。このように定義することにより、任意の引数と値を渡すことができます。

■ 呼び出すときの引数の書式

```
関数名(引数名 1 = 値 1, 引数名 2 = 値 2, ...)
```

辞書を引数に取る関数を呼び出す際は、dict 関数と同様に引数名（キー名）は文字列データとして扱われますが、引用符（「'～」や「"～」で囲うとエラーとなります。また、数字も指定できません。

■ 可変長引数（辞書）を持つ関数定義と呼び出しの例

```
def show_items2(**items):  
    print('「', items['str1'], '」')  
    print('「', items['str2'], '」')  
  
    for key, value in items.items():  
        print(key, ': ', value)  
  
show_items2(str1 = 'hoge', str2 = 'fuga')
```

【実行結果】

```
「 hoge 」  
「 fuga 」  
str1 : hoge  
str2 : fuga
```

show_items2()関数の引数である items は「辞書」です。要素の追加・更新・削除を自由に行うことができます。

7 章 ファイルを分割する

モジュール

Python 言語では関数の定義などがあるファイルと処理をするプログラムを分割することがよくあります。これは大規模なプログラムを開発するときには必須です。

この時、プログラムを**モジュール (module)** 単位に分割します。モジュールとは他のプログラムから利用できるようにしたプログラムです。

Python 言語のソースコードは厳密には**スクリプトファイル (script file)** と言い、モジュールはスクリプトファイルを分割するのに用います。

ファイルをモジュールに分割すると、スクリプトファイルの見通しが良くなります。

ファイルを分割しないと、一つのファイルが肥大化してプログラムが見づらくなり、生産性が低くなるだけでなく間違いやすくなります。

また、関連する機能をまとめられることもファイルを分割するメリットです。機能ごとにモジュールを分ければプログラムを整理しやすくなります。

更に、プログラムの管理が容易になり、担当者ごとにモジュールを割り当てることにより開発が容易になります。

モジュールを読み込む方法

同一フォルダ内で分割したファイルを読み込む方法には以下の 2 通りあります。

■ import 文を用いてモジュールを読み込む

```
import スクリプトファイル名
```

読み込むファイルが「**hoge.py**」なら、「**import hoge**」とします。「**.py**」という拡張子は省略してよいというルールがあります。

■ calc.py

```
def add(x,y):  
    return x + y
```

■ main.py

```
import calc  
  
ans = calc.add(2, 3)  
print(ans)
```

【実行結果】

```
5
```

実行するスクリプトは **main.py** です。

main.py では、「**import calc**」を用いて calc.py を読み込んでいるので、main.py 内で **add()**関数が呼び出されます。

モジュール内の関数を呼び出す時は「**モジュール名.関数名(引数)**」となります。

■ 特定のモジュールの特定の関数だけを読み込む

```
from スクリプトファイル名 import 関数名
```

以下に、サンプルを示します。

■ calc2.py

```
def add(x,y):  
    return x + y  
def sub(x,y):  
    return x - y
```

■ main2.py

```
import calc2  
from calc2 import add  
  
ans1 = add(2, 3)  
ans2 = calc2.sub(2, 3)  
print(ans1)  
print(ans2)
```

【実行結果】

```
5  
-1
```

calc2.py には **add()**関数と **sub()**関数が定義されています。

main2.py 内の「**from calc2 import add**」で calc2.py 内の **add()**関数を読み込み、「**add(2, 3)**」で関数を呼び出しています。

特定の関数を読み込んだ場合は**モジュール名が不要**です。

main2.py 内の「**import calc2**」で calc.py を読み込み、「**calc2.sub(2, 3)**」で関数を呼び出しています。**sub()**関数の呼び出しには**モジュール名が必要です**。

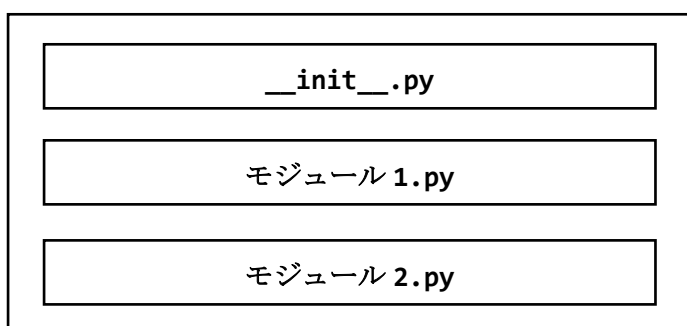
パッケージ

複数のモジュールを 1 つにまとめたものを **パッケージ (package)** といいます。

同じような機能を持つ複数のモジュールを 1 つのフォルダにまとめ、1 つの多機能なパッケージを作ります。

パッケージには、パッケージ名と呼ばれる名前がついています。「**パッケージ名=フォルダ名**」となります。内部に複数のモジュールのスクリプトファイルを配置するとともに、「**__init__.py**」というファイルを配置します。

■ パッケージの構成方法



「**__init__.py**」は、そのフォルダが **パッケージ** であることを示すファイルです。

中身が空でもかまいません。中身を記述する必要があるときはサブフォルダ等があるときで、設定をここに書きます。

パッケージの読み込み

__init__.py が空の場合にパッケージ内のモジュールを読み込む方法は「**import 文**」と「**from import 文**」の 2 通りあります。

まずは、**__init__.py** が空の場合から確認します。

■ import

```
import パッケージ名.モジュール名
```

■ import 時に関数を呼び出す書式

```
パッケージ名.モジュール名.関数名(引数)
```

■ from import

```
from パッケージ名 import モジュール名
```

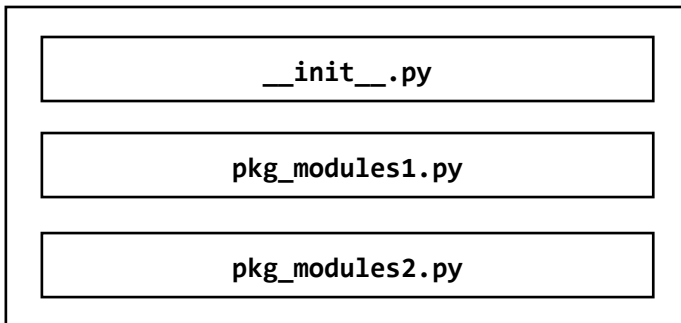
■ from import 時に関数を呼び出す書式

```
モジュール名.関数名(引数)
```

以下に `__init__.py` が空の場合のパッケージ読み込みのサンプルを示します。

`pkg` フォルダを作成して、それをパッケージとして利用します。`pkg` フォルダは以下のようになっています。

■ `pkg` フォルダ内の内容



※ `__init__.py` には、何も記述しません。

`pkg` フォルダ内に「`pkg_modules1.py`」と「`pkg_modules2.py`」を以下のように作成します。

■ `pkg_modules1.py`

```
def addItems(a,b):
    print(a+b)

def loop_func(loop):
    i = 0
    while i < loop:
        print(i)
        i = i + 1
```

■ `pkg_modules2.py`

```
def show_items(*data):
    for d in data:
        print(d)
```

pkg フォルダと同じ階層に「module3.py」を以下のように作成します。

※pkg フォルダの中ではありません。

■ module3.py

```
import pkg.pkg_modules1
from pkg import pkg_modules2

# 「import パッケージ名.モジュール名」でインポートした場合の呼び出し
pkg.pkg_modules1.addItem("hoge","fuga")
print("-" * 10)
pkg.pkg_modules1.loop_func(5)
print("-" * 10)
# 「from パッケージ名 import モジュール名」でインポートした場合の呼び出し
pkg_modules2.show_items('a','b','c')
```

【実行結果】

```
hogefuga
-----
0
1
2
3
4
-----
a
b
c
```

module3.py 内の「pkg.pkg_modules1.addItem("hoge","fuga")」と「pkg.pkg_modules1.loop_func(5)」は、「パッケージ名.モジュール名.関数名(引数)」の書式で関数を呼び出しています。

module3.py 内の「pkg_modules2.show_items('a','b','c')」は、「モジュール名.関数名(引数)」の書式で関数を呼び出しています。

■ as

パッケージやモジュールを **import** 後に使用する場合、名前が長く記述が煩雑になる場合があります。 **as** を使用することで、任意の別名を付けて短い名前でモジュールを扱えるようになります。

■ 書式

```
import ●● as 任意の名前
from ●● import ●● as 任意の名前
```

先ほどのサンプルを修正します。

```
import pkg.pkg_modules1 as m1
from pkg import pkg_modules2 as m2

# 「import パッケージ名.モジュール名」でインポートした場合の呼び出し
m1.addItem("hoge","fuga")
print("-" * 10)
m1.loop_func(5)
print("-" * 10)
# 「from パッケージ名 import モジュール名」でインポートした場合の呼び出し
m2.show_items('a','b','c')
```

実行結果は同じため割愛します。

また、パッケージの中のモジュールの中の関数を指定する場合、下記のような指定が可能です。

```
from pkg.pkg_modules2 import show_items
:
:
show_items('a','b','c')
```

`__init__.py` にパッケージ内のモジュールが指定されている場合のパッケージの読み込み方法を確認します。

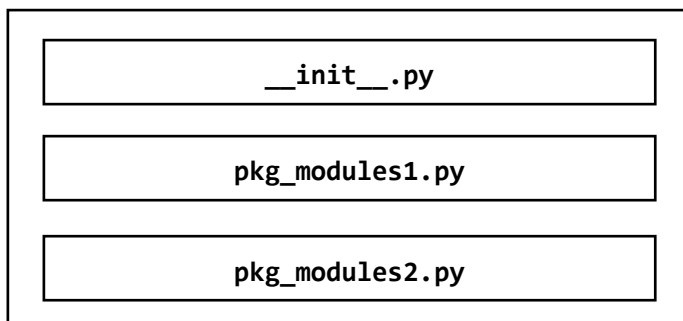
■ `__init__.py` にパッケージ内のモジュールが指定されている場合の `import` の書式

```
import パッケージ名
```

今回のパッケージは `pkg2` とします。

`pkg2` フォルダを作成して、「`__init__.py`」と「`pkg_module1.py`」と「`pkg_module2.py`」の3つのファイルを作成します。

■ `pkg2` フォルダ内の内容



※ `pkg_module1.py` と `pkg_module2.py` は `pkg` フォルダからコピーしてください。

`pkg2` フォルダ内に「`__init__.py`」を以下のように作成します。

■ `__init__.py`

```
from . import pkg_modules1
from . import pkg_modules2
```

「`.`」はカレントディレクトリを意味します。これは、カレントディレクトリ内（`pkg2`）の「`pkg_module1`」と「`pkg_module2`」を読み込むということを意味します。

pkg2 フォルダと同じ階層に「module4.py」を以下のように作成します。

■ module4.py

```
import pkg2

pkg2.pkg_modules1.addItem("abc","def")
print("-" * 10)
pkg2.pkg_modules1.loop_func(3)
print("-" * 10)
pkg2.pkg_modules2.show_items('x','y','z')
```

【実行結果】

```
abcdef
-----
0
1
2
-----
x
y
z
```

`__init__.py` にはカレントディレクトリにあるモジュールを読みに行く記述がされているので、「**import pkg2**」と記述するだけで、すべてのモジュールが自動的に読み込まれます。

このように様々なモジュールの読み方がありますが、**import** は全体を読み込む（使用しないモジュールも含まれる）**from import** は特定のモジュールや関数を読み込むという違いをおさえておきましょう。

8 章 オブジェクト指向

オブジェクト指向とは

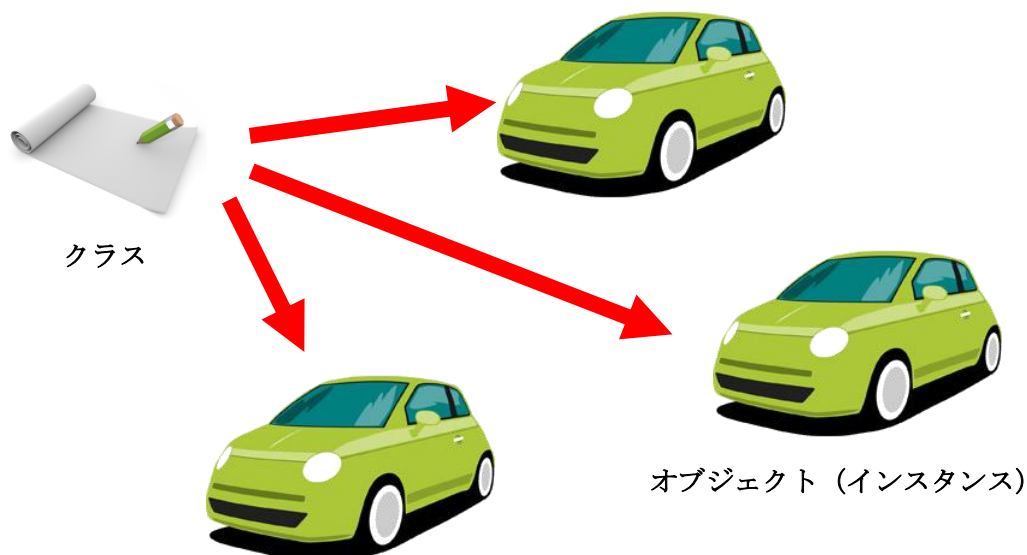
あらゆるものをすべて「オブジェクト(Object)」として表現するという考え方をオブジェクト指向と言います。現在主要なプログラミング言語のほとんどはオブジェクト指向です。Python 言語でもオブジェクト指向の概念は採用されています。

オブジェクト指向の考え方

前述のようにすべてをオブジェクトとして考えるのがオブジェクト指向です。オブジェクトには動作に当たるメソッドと、属性やデータにあたるメンバ変数が存在します。



オブジェクトはクラスをもとに生成されます。クラスとはオブジェクトの設計図のようなもので、クラスがあればいくつものオブジェクト（インスタンスとも言います）を生成できます。



一般にオブジェクト指向言語ではクラスを定義してオブジェクトを生成します。

クラスの定義

Python 言語では、クラスを以下のように定義します。

■ クラスの定義

```
class クラス名
    def __init__(self, 引数 1, 引数 2, ...):
        初期化メソッド（コンストラクタ）の処理

    def メソッド(self, 引数 1, 引数 2, ...):
        メソッドの処理
```

メソッドはクラス固有の関数で、第一引数には必ず **self** が入ります。

クラスには**初期化メソッド（コンストラクタ）**と呼ばれるメソッドがあります。これは、インスタンス生成時に一度だけ呼ばれるメソッドで、**__init__**という名前で定義します。

self とはインスタンス（自分自身）を表すキーワードで、「**self.変数名**」が**メンバ変数**と呼ばれる変数です。メンバ変数は、初期化メソッドで宣言します。

■ class01.py

```
class Person:
    def __init__(self):
        self.name = ""

    def getName(self):
        return self.name

    def setName(self, name):
        self.name = name

#インスタンスの生成と実行
p = Person()
p.setName("Taro")
person_name = p.getName()
print(person_name)
```

【実行結果】

```
Taro
```

`Person` クラス定義の後の処理は以下のような流れになっています。

1. インスタンスの生成

```
p = Person()
```

コンストラクタが実行され、メンバ変数 `self.name = ""` となります。

2. `setName()` メソッドの呼び出し

```
p.setName("Taro")
```

メンバー変数 `self.name` に `"Taro"` が代入されます。

3. `getName()` メソッドの呼び出し

```
person_name = p.getName()
```

メンバー変数 `self.name` の値を取得して、変数 `person_name` に代入します。

4. 変数 `person_name` の表示

```
print(person_name)
```

変数 `person_name` を表示します。 `"Taro"` と表示されます。

■ インスタンス生成の書式

```
変数 = クラス名(引数 1, 引数 2, ...)
```

■ メソッドを呼び出す書式

```
インスタンス名.メソッド名(引数 1, 引数 2, ...)
```

インスタンス名は、インスタンスが代入された変数名です。

複数のインスタンス

同じクラスから生成されてもインスタンスが違くと別物になります。メソッドの影響があるのはあくまでもそのインスタンス内のみです。

そのため、同じメソッド名でもインスタンスが異なると結果も異なります。メンバ変数も、インスタンスが違えば別物です。

つまり、`Person` クラスでインスタンスを複数生成しても、インスタンスが異なればメンバ変数の内容は異なります。

■ class02.py

```
class Person:
    def __init__(self):
        self.name = ""

    def getName(self):
        return self.name

    def setName(self, name):
        self.name = name

#インスタンスの生成と実行
p1 = Person()
p2 = Person()
p1.setName("Tom")
p2.setName("Bob")
print(p1.getName())
print(p2.getName())
```

【実行結果】

```
Tom
Bob
```

1つのクラスから複数のインスタンスを生成することができます。今回は、インスタンス p1、p2 を作り、setName()メソッドで"Tom"と"Bob"に設定します。

最後に getName()メソッドを呼び出し、インスタンス p1、p2 ではそれぞれ違う値が出力されているのがわかります。

継承

継承（けいしょう）はオブジェクト指向においてとても大事な概念です。継承は、あるクラスをもとにして更にその機能を拡張した新しいクラスを作ることです。

あるクラスを継承する新しいクラスを作るには、以下のようにします。

■ 継承して新しいクラスを作成する書式

```
class サブクラス名(スーパークラス名):  
    def メソッド(self, 引数 1, 引数 2, ...):  
        メソッドの処理
```

スーパークラスとは、元となるクラスのことです。スーパークラスを継承してできたクラスをサブクラスと言います。サブクラスは、スーパークラスのメンバ変数やメソッドを引継ぎ、さらに新しいものを追加することができます。

では実際に、サンプルを見てみましょう。

■ inherit.py

```
class Calc:  
    def __init__(self):  
        self.a = 0  
        self.b = 0  
  
    def add(self):  
        return self.a + self.b  
  
    def sub(self):  
        return self.a - self.b  
  
class SubCalc(Calc):  
    def mul(self):  
        return self.a * self.b  
  
    def div(self):  
        return self.a // self.b
```

ここでは、Calc クラスを継承した SubCalc クラスを作っています。次にこれらのクラスのインスタンスを生成して利用するサンプルを紹介します。

■ class03.py

```
from inherit import Calc
from inherit import SubCalc

sup = Calc()
sup.a = 1
sup.b = 2
print(sup.add())
print(sup.sub())

sub = SubCalc()
sub.a = 3
sub.b = 2
print(sub.add())
print(sub.sub())
print(sub.mul())
print(sub.div())
```

【実行結果】

```
3
-1
5
1
6
1
```

「from inherit import Calc」や「from inherit import SubCalc」のようにインポートすると、クラスも関数と同じように使用することができます。

SubCalc クラスは Calc クラスを継承しているため、自分自身で独自に定義したメソッドとともに、Calc クラスのメソッドを呼び出すことができます。そのため、sub は add()、sub()、mul()、div()という全メソッドを呼び出すことができます。

しかし、Calc クラスは add()と sub()メソッドしか定義していません。そのため、mul()と div()メソッドを呼び出そうとするとエラーが発生します。

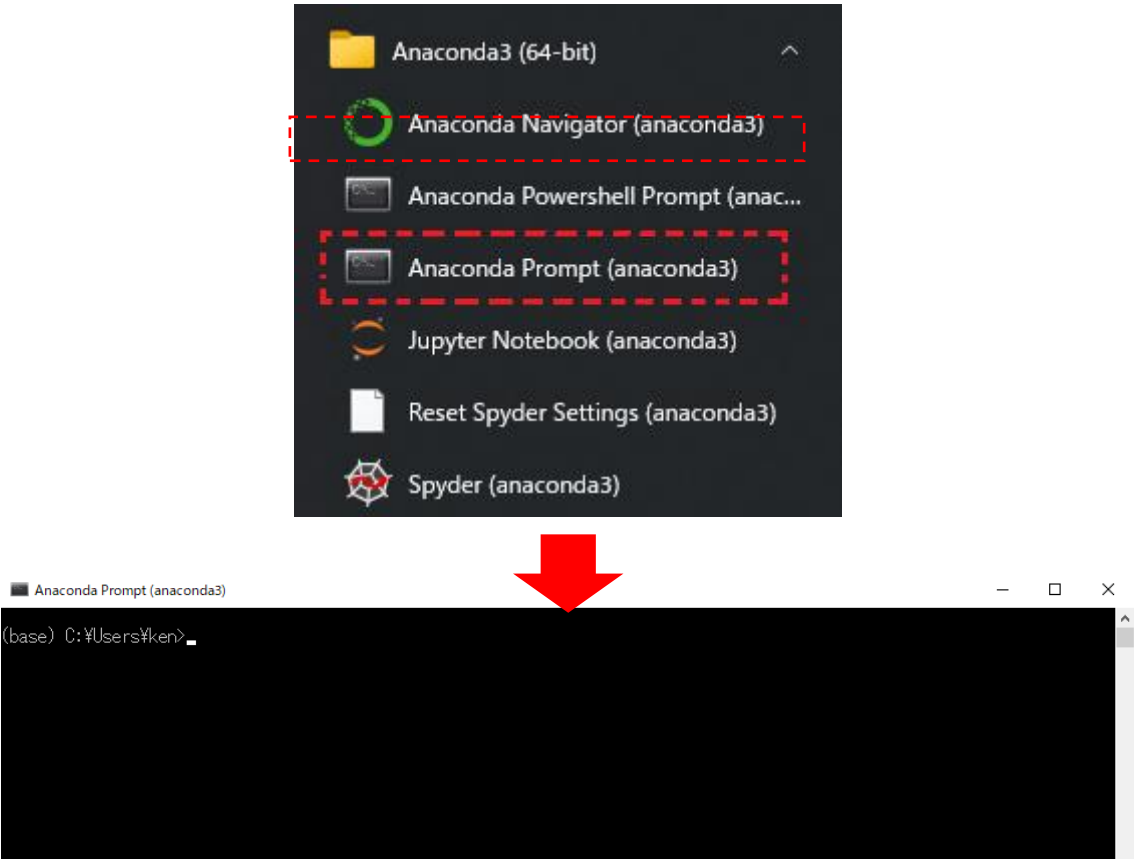
9 章 Python のライブラリ

pip コマンド

Python 言語には様々なライブラリが用意されています。しかし、通常それらのライブラリを利用することができません。外部のライブラリを利用するにはインストールが必要になります。

それを行うのが **pip** コマンドです。

Anaconda で **pip** を使うには、**Anaconda Prompt** を実行します。



pip を使ってパッケージをインストールするには、以下のような処理をします。

■ **pip** コマンドでパッケージをインストール

```
pip install <Package>
```

なお、**pip** には以下のような使い方があります。

処理	書式
pip install -U <Package>	パッケージのアップデート
pip uninstall <Package>	パッケージのアンインストール
pip search <Package>	指定したパッケージがあるかを探す
pip list	インストールされたパッケージの一覧を取得する
pip -V	pip のバージョンを取得

Matplotlib パッケージ

Matplotlib パッケージには様々なグラフを描くことができるモジュールが含まれています。ここでは、`matplotlib.pyplot` モジュールを利用して、グラフを描くための基本を学びます。

`matplotlib.pyplot` モジュールを利用するためにはインポートが必要です。その際に以下のように「`plt`」という別名をつけるのが一般的です。

```
import matplotlib.pyplot as plt
```

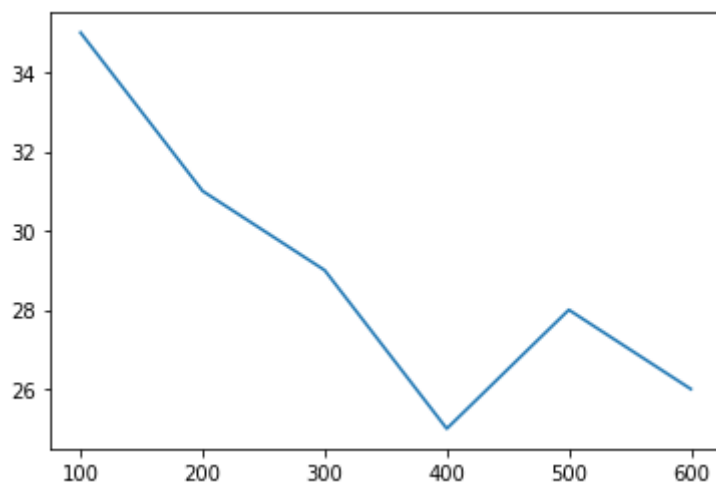
折れ線グラフ

それでは、`matplotlib.pyplot` モジュールを利用して、折れ線グラフを描く方法を学びます。

■ x 軸 price、y 軸 count の折れ線グラフを描画する例

```
import matplotlib.pyplot as plt
price = [100, 200, 300, 400, 500, 600]
count = [35, 31, 29, 25, 28, 26]
plt.plot(price, count)    # グラフを描く
plt.show()               # 描画する
```

【実行結果】

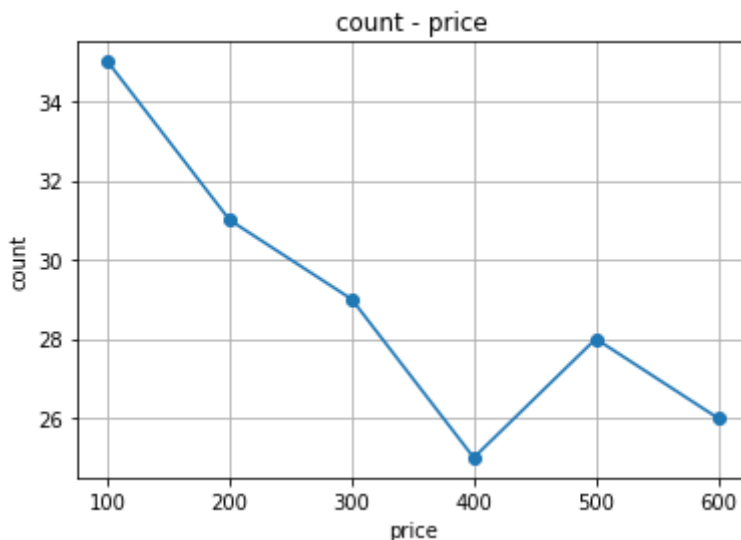


■ タイトル、軸ラベル、グリッド、マーカー付きの折れ線グラフを描画する例

```
import matplotlib.pyplot as plt
price = [100, 200, 300, 400, 500, 600]
count = [35, 31, 29, 25, 28, 26]
plt.plot(price, count, marker="o")    # グラフを描く（マーカー付き）
plt.title("count - price")           # タイトル
plt.xlabel("price")                   # x 軸のラベル
plt.ylabel("count")                   # y 軸のラベル
plt.grid(True)                        # グリッド
plt.show()                            # 描画する
```

※「marker="o"」のように指定すると、丸いマーカーになります。

【実行結果】



グラフを保存する

`savefig()`関数を利用すると、描画した画像を保存することができます。画像ファイルはカレントディレクトリに保存されます。

```
plt.savefig("ファイル名")
```

`savefig()`関数で画像を保存するときは、必ず `show()`関数よりも先に実行しなければいけません。画像の保存と描画を行いたい時は、まず `savefig()`関数を呼び出し、その後 `show()`関数を呼び出してください。

■ 折れ線グラフを保存し、描画する例

それでは先程作成したプログラムを下記のように修正します。

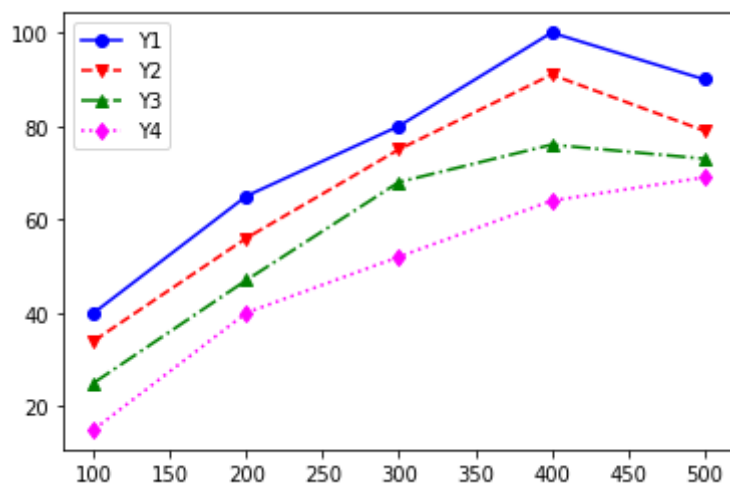
```
import matplotlib.pyplot as plt
price = [100, 200, 300, 400, 500, 600]
count = [35, 31, 29, 25, 28, 26]
plt.plot(price, count, marker="o")    # グラフを描く（マーカー付き）
plt.title("count - price")           # タイトル
plt.xlabel("price")                   # x 軸のラベル
plt.ylabel("count")                   # y 軸のラベル
plt.grid(True)                        # グリッド
plt.savefig("graph01.png")            # 保存する
plt.show()                            # 描画する
```

※【実行結果】は変わらないので割愛します。

■ 凡例付きで、折れ線の種類やマーカーが違うグラフを描画する例

```
import matplotlib.pyplot as plt
X = [100, 200, 300, 400, 500]
Y1 = [40, 65, 80, 100, 90]
Y2 = [34, 56, 75, 91, 79]
Y3 = [25, 47, 68, 76, 73]
Y4 = [15, 40, 52, 64, 69]
plt.plot(X, Y1, marker="o", color="blue", linestyle="solid", label="Y1")
plt.plot(X, Y2, marker="v", color="red", linestyle="dashed", label="Y2")
plt.plot(X, Y3, marker="^", color="green", linestyle="dashdot", label="Y3")
plt.plot(X, Y4, marker="d", color="magenta", linestyle="dotted", label="Y4")
plt.legend(loc="upper left")
plt.show()
```

【実行結果】



■ マーカー (marker) の種類

マーカーにはたくさんの種類があり、一部のみ紹介します。

マーカーは、丸 ("o")、星 ("*")、菱形 ("D")、薄い菱形 ("d")、三角形下向き ("v")、三角形上向き ("^")、三角形左向き ("<")、三角形右向き (">")などを指定することができます。

■ 色の種類 (color) の種類

線の色は自動で設定されますが、"red"、"green"、"blue"、"yellow"、"cyan"、"magenta"、"white"、"black"、カラーコード (例: #7ac2ff) で指定できます。

■ 線 (linestyle) の種類

線の種類は、実線 ("solid")、破線 ("dashed")、一点鎖線 ("dashdot")、点線 ("dotted")を指定することができます。

また線の太さは、「linewidth=2.5」のように指定できます。

■ 凡例の表示位置

legend()関数を利用すると、凡例を表示することができます。引数 loc でどの位置に表示するのかを指定することができます。

表示位置は、右上 ("upper right")、左上 ("upper left")、右下 ("lower right")、左下 ("lower left")、右 ("right")、中央右 ("center right")、中央左 ("center left")、中央上 ("upper center")、中央下 ("lower center")、中央 ("center") を指定することができます。

matplotlib.pyplot モジュールでは、折れ線グラフ以外に棒グラフや散布図なども描画することができます。plot()関数以外にも調べて試してみましょう。

NumPy ライブラリ

リストと NumPy ライブラリの配列は共通した機能が多くあります。しかし、NumPy ライブラリの配列は多次元配列に対する処理が非常に優れており、数値データを扱う機能が豊富で高速な計算ができます。

データ分析や機械学習・人工知能のプログラムを作成するには膨大な量のデータを必要とします。通常収集した状態のデータはそのままでは使用できないことが多く、データクレンジング（データの中から、削除や修正、正規化などを行ってデータの品質を高め加工すること）を行います。また、機械学習・人工知能のライブラリには NumPy のデータを引数に取る機能が多くあります。

ここでは、NumPy ライブラリの配列を利用して、多次元配列の作り方とデータ操作を学びます。

NumPy ライブラリを利用するためにはインポートが必要です。その際に以下のように「np」という別名をつけるのが一般的です。

```
import numpy as np
```

NumPy の配列

NumPy の配列は、`array()`関数を利用すると作成できます。配列の要素は、リストまたはタプルで指定します。

■ リストとタプルから NumPy の配列を作成する例

```
import numpy as np
arr1 = np.array([1,2,3])
arr2 = np.array((4,5,6))
print(arr1)
print(arr2)
```

【実行結果】

```
[1 2 3]
[4 5 6]
```

リストの表示結果「`[1, 2, 3]`」とは異なり、NumPy の配列は「`,`」が表示されません。

■ データ型が混在している NumPy の配列の作成例

加工前のデータには様々な種類の情報が含まれていることもあります。

numpy では様々なデータを配列としてまとめることが可能で、基本的に大きなデータ型に合わせて変換されます

```
import numpy as np
arr1 = np.array([1, 1.5, 2])           #int と float
print(arr1)    #すべて float
arr2 = np.array([1, True, False])      #int と bool
print(arr2)    #すべて int
arr3 = np.array([1, 1.5, True, False]) #int と float と bool
print(arr3)    #すべて float
arr4 = np.array([1, 1.5, "2"])          #int と float と str
print(arr4)    #すべて str
```

【実行結果】

```
[1.  1.5  2. ]
[1  1  0]
[1.  1.5  1.  0. ]
['1' '1.5' '2']
```

■ データ型を指定した NumPy 配列の作成例

dtype 引数を設定することで、任意のデータ型を指定することができます。

```
import numpy as np
arr1 = np.array([1, 1.5, 2], dtype=int)   #int を指定
print(arr1)
arr2 = np.array([1, 2, 3], dtype=float)   #float を指定
print(arr2)
arr3 = np.array([1, 1.5, 2], dtype=str)   #str を指定
print(arr3)
arr4 = np.array(arr1, dtype=float)        #float を指定
print(arr4)
```

【実行結果】

```
[1  1  2]
[1.  2.  3.]
['1' '1.5' '2']
[1.  1.  2.]
```

NumPy の多次元配列

NumPy は多次元配列（多次元行列）を扱うのに優れています。

■ リストから NumPy の多次元配列を作成する例

```
import numpy as np
arr1 = np.array([[1, 2, 3],[4, 5, 6]])
print(arr1)
l1=[10, 20, 30]
l2=[40, 50, 60]
l3=[70, 80, 90]
arr2=np.array([l1,l2,l3])
print(arr2)
```

【実行結果】

```
[[1 2 3]
 [4 5 6]]
[[10 20 30]
 [40 50 60]
 [70 80 90]]
```

配列データの構造確認

データ分析では、データの形状、データ型、大きさなどデータの構造を調査し把握する必要があります。

shape	形状
ndim	次元数
dtype	データ型
size	大きさ

先ほどのサンプルコードに下記を追加して確認しましょう。

```
import numpy as np
arr1 = np.array([[1, 2, 3],[4, 5, 6]])
print(arr1.shape)
print(arr1.ndim)
print(arr1.dtype)
print(arr1.size)
```

【実行結果】

```
(2, 3) ←arr1 の形状 2 行 3 列
2      ←arr1 の次元数
int32  ←arr1 のデータ型
6      ←arr1 のサイズ
```

次元の変換

機械学習・人工知能のライブラリの中には、次元数を揃えて使用する場合があります。
次元数を変換するには `reshape(行,列)` メソッドを使用します。
※`reshape` メソッドの引数は次元数に応じて変わります。

```
import numpy as np

arr = np.array(range(0,12)) ← 1次元配列
print(arr)
arr2_6 = arr.reshape(2, 6) ← 2次元配列に変換
print(arr2_6)

arr = arr2_6.reshape(12) ← 2次元から1次元に変換
print(arr)
print(arr.reshape(2,2,3)) ← 1次元から3次元に変換
```

【実行結果】

```
[ 0  1  2  3  4  5  6  7  8  9 10 11] ← 1次元配列
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]] ← 2次元配列
[ 0  1  2  3  4  5  6  7  8  9 10 11] ← 1次元配列
[[[ 0  1  2]
 [ 3  4  5]]
 [[ 6  7  8]
 [ 9 10 11]]] ← 3次元配列
```

要素の追加

NumPy の配列に要素を追加するときは、`append()`関数を利用します。`append()`関数は `array()`関数と同様に要素が追加された新しい配列を作成します。

■ `append()`関数の使用例

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.append(arr1, 4)
print(arr1)    #配列 arr1 は変化しない
print(arr2)    #新しい配列 arr2 が作られる
arr3 = np.append(arr1, [4, 5, 6])          #3 個追加
print(arr3)    #配列 arr1 に 3 個追加された新しい配列 arr3 が作られる
arr4 = np.array([[1, 2, 3],[4, 5, 6]])
arr5 = np.append(arr4, [[7, 8, 9]], axis=0) #配列 arr4 に新規行を追加
print(arr4)
print(arr5)
```

`append()`関数の引数に「`axis=0`」を指定すると、行方向に追加され、「`axis=1`」を指定すると、列方向に追加されます。指定しない場合は、2次元配列が1次元配列になります。

【実行結果】

```
[1 2 3]      ←配列 arr1 は変化しない
[1 2 3 4]     ←新しい配列 arr2
[1 2 3 4 5 6] ←配列 arr1 に 3 個追加された配列 arr3
[[1 2 3]
 [4 5 6]]    ←配列 arr4
[[1 2 3]
 [4 5 6]
 [7 8 9]]    ←配列 arr4 に [7 8 9] が追加された配列 arr5
```

■ `append()`を使用して、列を追加する場合

```
arr5 = np.append(arr4, [[7],[8]], axis=1)
```

要素の挿入

NumPy の配列に要素を挿入するときは、`insert()`関数を利用します。`insert()`関数は `array()`関数と同様に要素が挿入された新しい配列を作成します。

■ `insert()`関数の使用例

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.insert(arr1, 1, 99)    #インデックス 1 の位置に 99 を挿入
print(arr1)    #配列 arr1 は変化しない
print(arr2)    #新しい配列 arr2 が作られる
arr3 = np.insert(arr1, 1, [40, 50])    #インデックス 1 の位置に 2 個挿入
print(arr3)    #インデックス 1 の位置に 2 個挿入された新しい配列 arr3 が作られる
arr4 = np.array([[1, 2, 3],[4, 5, 6]])
arr5 = np.insert(arr4, 0, [[7, 8, 9]], axis=0)    #配列 arr4 に新規行を挿入
print(arr4)
print(arr5)
```

`insert()`関数の引数に「`axis=0`」を指定すると、行方向に追加され、「`axis=1`」を指定すると、列方向に追加されます。指定しない場合は、2 次元配列が 1 次元配列になります。

【実行結果】

```
[1 2 3]          ←配列 arr1 は変化しない
[ 1 99  2  3]     ←新しい配列 arr2 が作られる
[ 1 40 50  2  3]  ←インデックス 1 の位置に 2 個挿入された配列 arr3
[[1 2 3]
 [4 5 6]]         ←配列 arr4
[[7 8 9]
 [1 2 3]
 [4 5 6]]         ←配列 arr4 のインデックス 0 の位置に [7 8 9] が挿入された配列 arr5
```

要素へのアクセスと値の更新

インデックス番号を使用した要素へのアクセスと値の更新、配列のスライスを確認します。

■ インデックス番号を使用した要素へのアクセスの例

```
import numpy as np
arr1 = np.array([1, 2, 3, 4, 5])
print(arr1[2])
print(arr1[-2])
arr1[1] = 99          #arr1[1]の値を更新
print(arr1)
arr2 = np.array([[1, 2, 3],[4, 5, 6]])
print(arr2[0, 0])    #arr2[0][0]と同じ
print(arr2[1, 2])    #arr2[1][2]と同じ
print(arr2[-1, -2])  #arr2[-1][-2]と同じ、この配列では arr2[1][1]と同じ
arr2[0, 2] = 99      #arr2[0, 2]の値を更新
print(arr2)
```

【実行結果】

```
3
4
[ 1 99  3  4  5]
1
6
5
[[ 1  2 99]
 [ 4  5  6]]
```

■ 配列のスライスの例

```
import numpy as np
arr1 = np.array(range(0,10))
arr2 = arr1[:]          #すべての要素（新しい配列が作られる）
arr3 = arr1[:4]         #インデックス 0 から 3 までの配列
arr4 = arr1[4:]         #インデックス 4 から最後まで
arr5 = arr1[3:7]        #インデックス 3 から 6 までの配列
arr6 = arr1[: :2]       #インデックス番号が偶数の配列
arr7 = arr1[: :-1]      #末尾から 1 つずつ取得した配列
print(arr1)
print(arr2)
print(arr3)
print(arr4)
print(arr5)
print(arr6)
print(arr7)
```

【実行結果】

```
[0 1 2 3 4 5 6 7 8 9] ←スライスしていない状態の配列
[0 1 2 3 4 5 6 7 8 9] ←すべての要素（新しい配列が作られる）
[0 1 2 3]              ←インデックス 0 から 3 までの配列
[4 5 6 7 8 9]          ←インデックス 4 から最後まで
[3 4 5 6]              ←インデックス 3 から 6 までの配列
[0 2 4 6 8]            ←インデックス番号が偶数の配列
[9 8 7 6 5 4 3 2 1 0] ←末尾から 1 つずつ取得した配列
```

■ 多次元配列のスライスの例

```
import numpy as np
arr1 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
arr2 = arr1[0:2 , 0:2]    #0 から 1 行目の 0 から 1 列目までの配列
arr3 = arr1[:, 1:2]      #すべての行の 2 列目の配列
arr4 = arr1[1: , 1:]      #1 行目以降の 1 列目以降の配列
print(arr1)
print(arr2)
print(arr3)
print(arr4)
```

【実行結果】

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]    ←スライスしていない状態の多次元配列
[[1 2]
 [4 5]]      ←0 から 1 行目の 0 から 1 列目までの配列
[[2]
 [5]
 [8]]        ←すべての行の 2 列目の配列
[[5 6]
 [8 9]]      ←1 行目以降の 1 列目以降の配列
```

要素の抽出と並べ替え

■ すべての要素を抽出する例

```
import numpy as np
arr1 = np.array(range(10,51,10)) #range(開始値,終了値,増分値)
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
for item in arr1:
    print(item)
print() #改行
for index, item in enumerate(arr1, 0):
    print(index, "=", item)
print() #改行
for index, item in np.ndenumerate(arr2):
    print(index, "=", item)
```

NumPy の配列は、リストと同じように繰り返し処理ができます。enumerate()を使用すると、繰り返す回数（インデックス）を取得することができます。

NumPy の多次元配列の要素を 1 つずつアクセスするときは、ndenumerate()を使用します。今回の例では、「index」に「(行番号, 列番号)のタプル」が入り、「item」に「要素」が入ります。

【実行結果】

```
10
20
30
40
50

0 = 10
1 = 20
2 = 30
3 = 40
4 = 50

(0, 0) = 1
(0, 1) = 2
(0, 2) = 3
(1, 0) = 4
(1, 1) = 5
(1, 2) = 6
```

■ 条件式で要素を抽出する例

```
import numpy as np
arr1 = np.array(range(1,10))
arr2 = arr1[arr1 >= 5]          #5 以上の要素を抽出
arr3 = arr1[arr1 % 2 == 0]      #偶数を抽出
arr4 = arr1[arr1 % 2 == 1]      #奇数を抽出
arr5 = arr1[(arr1 >= 5) & (arr1 % 2 == 1)]      #5 以上の奇数を抽出
arr6 = arr1[(arr1 % 2 == 0) | (arr1 % 3 == 0)]    #2 と 3 の倍数を抽出
arr7 = arr1.reshape(3,3)
arr8 = arr7[arr7 % 2 == 1]      #奇数を抽出
print(arr1)
print(arr2)
print(arr3)
print(arr4)
print(arr5)
print(arr6)
print(arr7)
print(arr8)
```

配列の中から条件式で要素を抽出するとき、論理演算を行うには、「&」、「|」、「~」を利用します。「and」、「or」、「not」は利用できません。

【実行結果】

```
[1 2 3 4 5 6 7 8 9]  ←スライスしていない状態の配列
[5 6 7 8 9]          ←5 以上の要素を抽出
[2 4 6 8]            ←偶数を抽出
[1 3 5 7 9]          ←奇数を抽出
[5 7 9]              ←5 以上の奇数を抽出
[2 3 4 6 8 9]        ←2 と 3 の倍数を抽出
[[1 2 3]
 [4 5 6]
 [7 8 9]]            ←1 次元から 2 次元に reshape した状態の多次元配列
[1 3 5 7 9]          ←奇数の要素を抽出
```

■ 配列をソートする例

```
import numpy as np
arr1 = np.array([3, 1, 4, 2, 5, 0])
arr1.sort()      #arr1 を昇順にソート
arr2 = np.sort([9, 2, 6, 5, 3, 8])      #リストから昇順にソートした
arr3 = np.sort([5, 8, 1, 7, 9, 3])[: :-1]  #降順にソート
print(arr1)
print(arr2)
print(arr3)
```

【実行結果】

```
[0 1 2 3 4 5]    ←arr1 を昇順にソート
[2 3 5 6 8 9]    ←リストから昇順にソートした配列を作成
[9 8 7 5 3 1]    ←降順にソート
```

配列の演算

NumPy の配列は、全要素に対して数値演算や比較演算を行うことができます。

■ 数値演算の例

```
import numpy as np
arr1 = np.array([[10, 20, 30], [40, 50, 60]])
arr2 = arr1 + 6
arr3 = arr1 - 5
arr4 = arr1 / 4
arr5 = arr1 % 4
arr6 = arr1 * 3
arr7 = arr1 ** 2
print(arr1)
print(arr2)
print(arr3)
print(arr4)
print(arr5)
print(arr6)
print(arr7)
```

【実行結果】

```

[[10 20 30]
 [40 50 60]]      ←演算前のすべての要素
[[16 26 36]
 [46 56 66]]      ←すべての要素に 6 を足した配列
[[ 5 15 25]
 [35 45 55]]      ←すべての要素から 5 を引いた配列
[[ 2.5  5.  7.5]
 [10.  12.5 15. ]] ←すべての要素を 4 で割った配列
[[2 0 2]
 [0 2 0]]          ←すべての要素を 4 で割った余りの配列
[[ 30  60  90]
 [120 150 180]]    ←すべての要素に 3 を掛けた配列
[[ 100  400  900]
 [1600 2500 3600]] ←すべての要素を 2 乗した配列

```

■ 配列の合計、最大値、最小値、平均値の求める例

データ分析では様々な集計を必要とします。ここでは、NumPy に用意されている代表的なメソッドを紹介します。

```

import numpy as np
arr1 = np.array([[28, 56, 49],[93, 75, 31]])
print(arr1)
print()    #改行
print(arr1.sum())    #全要素の合計
print(arr1.sum(0))   #各列の合計
print(arr1.sum(1))   #各行の合計
print()    #改行
print(arr1.max())    #全要素の最大値
print(arr1.max(0))   #各列の最大値
print(arr1.max(1))   #各行の最大値
print()    #改行
print(arr1.min())    #全要素の最小値
print(arr1.min(0))   #各列の最小値
print(arr1.min(1))   #各行の最小値
print()    #改行
print(arr1.mean())   #全要素の平均
print(arr1.mean(0))  #各列の平均
print(arr1.mean(1))  #各行の平均

```

【実行結果】

```
[[28 56 49]
 [93 75 31]]

332          ←全要素の合計
[121 131 80] ←各列の合計
[133 199]    ←各行の合計

93           ←全要素の最大値
[93 75 49]   ←各列の最大値
[56 93]      ←各行の最大値

28           ←全要素の最小値
[28 56 31]   ←各列の最小値
[28 31]      ←各行の最小値

55.33333333333336 ←全要素の平均
[60.5 65.5 40. ] ←各列の平均
[44.33333333 66.33333333] ←各行の平均
```

■ 比較演算の例

```
import numpy as np
arr1 = np.array(range(1,10))
arr2 = (arr1 > 5)      #5 より大きい要素の位置を判定
print(arr2)            #5 より大きい要素の位置は True
print(arr2.sum())      #5 より大きい要素の総数
```

【実行結果】

```
[False False False False False  True  True  True  True]
4
```

■ 配列同士の数値演算の例

NumPy の配列は同じ行列数ならば、配列同士で演算することができます。

```
import numpy as np
arr1 = np.array([10, 20, 30, 40, 50])
arr2 = np.array([4, 2, 5, 9, 6])
arr3 = arr1 + arr2    #arr1 と arr2 の各要素を足し算した配列
arr4 = arr1 - arr2    #arr1 と arr2 の各要素を引き算した配列
arr5 = arr1 * arr2    #arr1 と arr2 の各要素を掛け算した配列
arr6 = arr1 / arr2    #arr1 と arr2 の各要素を割り算した配列
print(arr3)
print(arr4)
print(arr5)
print(arr6)
```

【実行結果】

```
[14 22 35 49 56]
[ 6 18 25 31 44]
[ 40 40 150 360 300]
[ 2.5      10.      6.      4.44444444  8.33333333]
```


Python 基礎研修課題

BMI 判定問題

■ BMI の算出

$$\text{BMI} = \text{体重(kg)} \div \text{身長(m)}^2$$

下記のような結果を満たすプログラムを作成してください。

- 身長と体重を入力させます
- 入力値をもとに BMI を算出します
- 算出した BMI が 18.5未満の場合は「やせている」と表示します
- 算出した BMI が 18.5 以上 25未満の場合は「普通」と表示します
- 算出した BMI が 25 以上 35未満の場合は「やや肥満」と表示します
- 算出した BMI が 35 以上の場合は「肥満」と表示します

■ 実行例 1

体重(kg):80

身長(m):1.80

普通(BMI:24.691358024691358)

■ 実行例 2

体重(kg):70

身長(m):1.60

やや肥満(BMI:27.343749999999996)

くじ引き問題

1～100 までの数値をランダムに取得して、3 の倍数と 3 のつく数値のときだけ「当たり」、それ以外の数値は「はずれ」と表示されるプログラムを作成してください。

■ 1～5 までの数値をランダムに取得する例

```
import random

num = random.randint(1, 5)
```

■ "A"が含まれているか判定する例

```
text = "ABC"

print("A" in text)    #True
print("X" in text)    #False
```

■ 実行例 1

```
くじ引き:31
当たり
```

■ 実行例 2

```
くじ引き:98
はずれ
```

■ 実行例 3

```
くじ引き:84
当たり
```

RPG のバトルシーン問題

有名な RPG のバトルシーンをプログラムで作成します。

下記のような結果を満たすプログラムを作成してください。

- ゆうしゃのヒットポイントは **15** とします
- スライムのヒットポイントは **8** とします
- 攻撃はゆうしゃから始まり、スライムと交互に攻撃をします
- 攻撃は0～7までの数値をランダムに取得してその値を利用します。
- 攻撃が0の時は、下記を表示してください。
「ミス！○○○○にダメージをあたえられない！」

■ 実行例 1

```
スライム に 1 のダメージ  
ゆうしゃ に 5 のダメージ  
スライム に 1 のダメージ  
ゆうしゃ に 1 のダメージ  
スライム に 2 のダメージ  
ミス！ ゆうしゃ にダメージをあたえられない！  
スライム に 7 のダメージ  
スライムをたおした
```

■ 実行例 2

```
スライム に 1 のダメージ  
ゆうしゃ に 6 のダメージ  
スライム に 3 のダメージ  
ゆうしゃ に 5 のダメージ  
スライム に 1 のダメージ  
ゆうしゃ に 3 のダメージ  
ミス！ スライム にダメージをあたえられない！  
ゆうしゃ に 2 のダメージ  
ゆうしゃはまけた
```

自作関数問題

スクリプトファイル「myfunctions.py」を作成して、以下の関数を定義してください。

問題①

■ func_sum()関数の定義

```
func_sum(**kargs)
```

可変長引数（辞書）を持つ関数定義方法を利用して、**start** から **end** までの合計を返す関数を作成してください。

func_sum()関数が完成したら、呼び出すコードを作成してください。

■ 実行例

```
# func_sum()関数を呼び出した結果、引数に start=1, end=20 を指定。  
210
```

問題②

■ make_random_list()関数の定義

```
make_random_list(size)
```

0~100 までのランダムな整数が代入されたリストを返します。

引数には戻り値であるリストの要素数です。

make_random_list()関数が完成したら、呼び出すコードを作成してください。

■ 実行例

```
# make_random_list()関数を呼び出した結果、引数に 10 を指定  
[75, 14, 9, 82, 82, 57, 88, 27, 91, 54]
```


Python 基礎研修課題解答例

BMI 判定問題 解答例

```
weight = input("体重(kg):")
height = input("身長(m):")

w = float(weight)
h = float(height)
bmi = w / h ** 2;

if bmi < 18.5:
    result = "やせている"
elif bmi < 25:
    result = "普通"
elif bmi < 35:
    result = "やや肥満"
else:
    result = "肥満"

print(result + "(BMI:" + str(bmi) + ")")
```

くじ引き問題 解答例

```
import random

num = random.randint(1, 100)
print("くじ引き:" + str(num))

if num % 3 == 0:
    print("当たり")
elif "3" in str(num):
    print("当たり")
else:
    print("はずれ")
```

RPG のバトルシーン問題 解答例

```
import random

hero = 15
slime = 8
i = 0
while hero > 0 and slime > 0:
    attack = random.randint(0, 7)
    if i % 2 == 0:
        slime -= attack
        target = "スライム"
    else:
        hero -= attack
        target = "ゆうしゃ"

    if attack == 0:
        print("ミス!", target, "にダメージをあたえられない!")
    else:
        print(target, "に", attack, "のダメージ")

    i += 1

if hero <= 0:
    print("ゆうしゃはまけた")
else:
    print("スライムをたおした")
```

自作関数問題 解答例

myfunctions.py

```
import random

def func_sum(**kargs):
    start = kargs["start"]
    end = kargs["end"]
    result = 0
    i = start
    while i <= end:
        result += i
        i += 1
    print(result)

def make_random_list(size):
    result = []
    for i in range(size):
        num = random.randint(0, 100)
        result.append(num)
    return result
```

func_sum()関数と make_random_list()関数を呼び出すコード

```
import myfunctions

myfunctions.func_sum(start=1, end=20)

print(myfunctions.make_random_list(10))
```

Appendix

クラウド型実行環境【Colaboratory】

Google Colaboratory（以下、Colab）は、Google が無料で提供している、ブラウザから Python を記述・実行できるサービスです。

環境構築が不要・GPU への無料アクセス・共有が簡単、といった特長を備えています。



Colab では、Google アカウントがあれば簡単に Python の実行環境を構築できます。機械学習で必要になる NumPy などの外部ライブラリもインストール済みです。また、機械学習などの重い処理が高速に行える GPU も搭載されています。

Colab には有料版も用意されています。

名称	利用料	GPU	メモリ	長時間ランタイム	バックグラウンド実行
Colab	無料	△	×	×	×
Colab Pro	¥1,072-	○	○	○	×
Colab Pro+	¥5,243-	◎	◎	◎	○

有料版の方が無料版よりも高スペックですが、個人で試す分には無料版でも十分なスペックです。

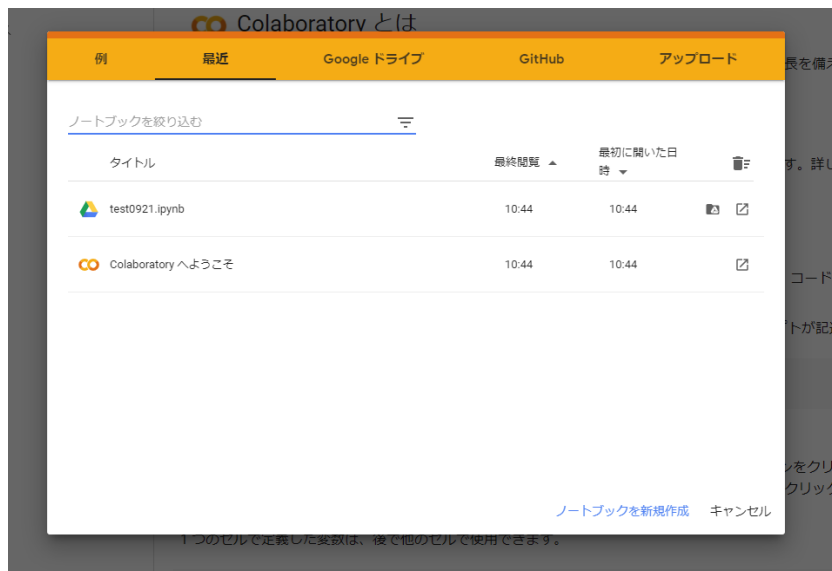
ファイルの準備

- ① 公式サイトにアクセスします。

<https://colab.research.google.com/notebooks/intro.ipynb?hl=ja>

- ② Google アカウントでログインします。

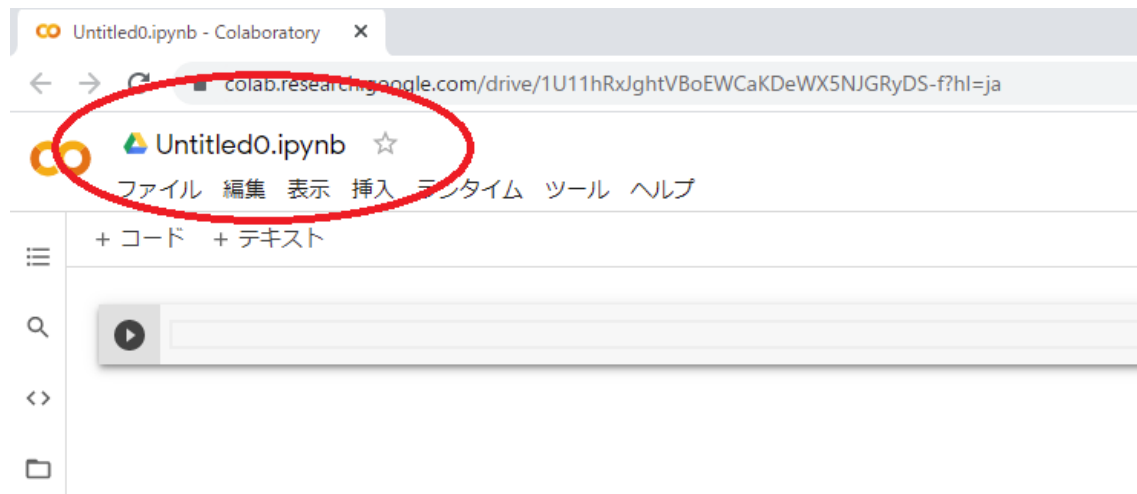
※、初回以外のログイン時はファイル選択が求められます。



- ③ 「ファイル」→「ノートブックを新規作成」



- ④ 「Untitled0.ipynb」をクリックするとノートブック名を編集できます。



- ⑤ ファイルは「ファイル」→「ドライブで探す」→「マイドライブ」(Google ドライブ) →「Colab Notebooks」に保存されます。

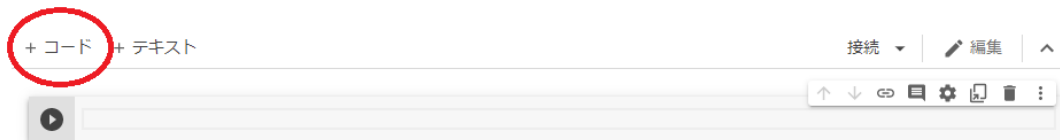
コードセルの編集と実行

ノートブックには、コードセルとテキストセルがあります。

コードセル : Python でコードを記述する場所

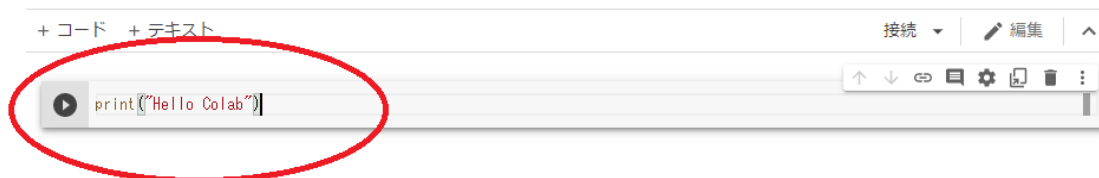
テキストセル : メモを残す場所 (※、コメントとは異なります)

- ① 「+コード」をクリックしコードセルの追加を行います。

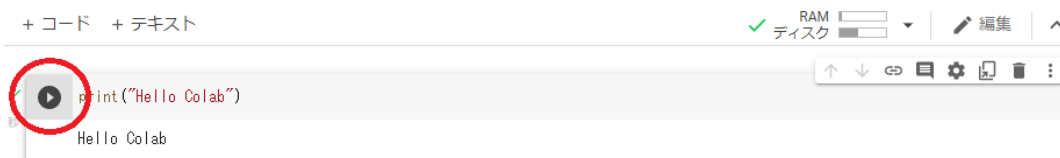


- ② 追加されたコードセルにコードを入力します。

例 : `print("Hello Colab")`

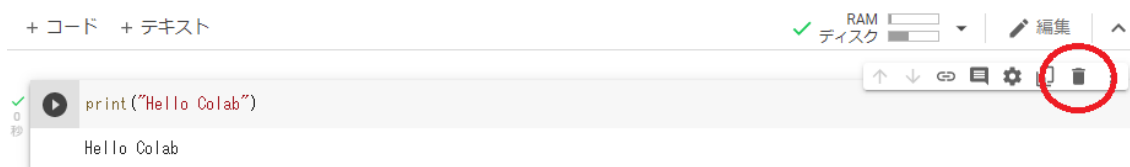


- ③ コードセルの実行ボタンをクリックすると実行されます。



※ 実行が止まらない場合は、同じボタンで停止できます。

- ④ コードセルをクリックすると再編集が可能です。
⑤ 不要になったコードセルや出力は、「右クリック」→「セルの削除」もしくは「クリック」→「ゴミ箱のアイコン」をクリックで削除できます。



- ⑥ 作成したファイルは「ファイル」→「ダウンロード」→「.py をダウンロード」で PC にダウンロードできます。
⑦ 別のノートブックを開きたい場合は、「ファイル」→「ノートブックをアップロード」からファイルを指定します。

テキストセルの編集

- ① 「+テキスト」をクリックしテキストセルの追加を行います。



- ② 追加されたテキストセルにテキストを入力します。

「#」で始まるテキストは見出しになります。

「-」（ハイフン・半角スペース）で始まるテキストは箇条書きになります。



- ③ 再編集は「右クリック」→「編集」もしくは「クリック」→「鉛筆のアイコン」をクリックで行います。
- ④ コードセルやテキストセルの位置を変更する場合は、対象のセルをクリック後、メニューの上下の矢印アイコンで行います。

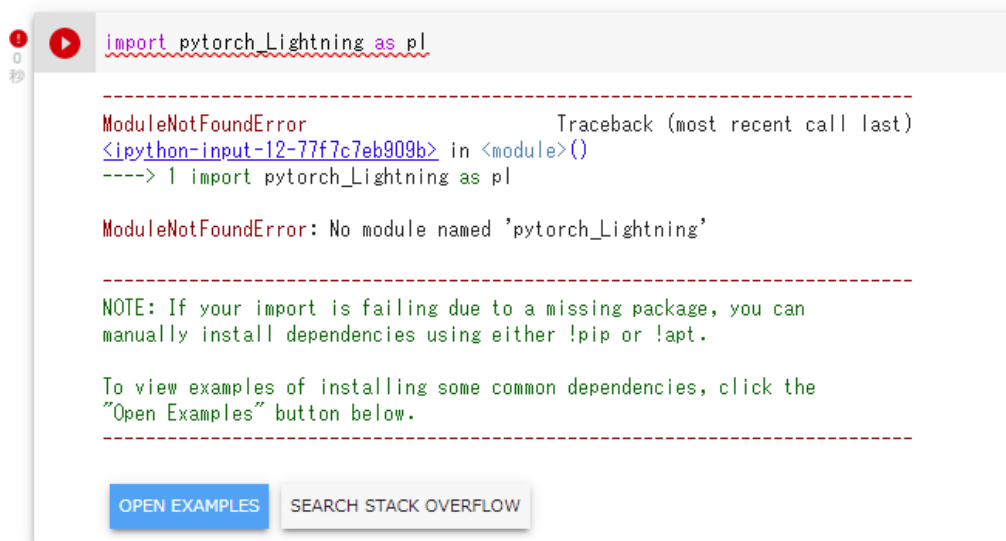
外部ファイルの準備

画像や CSV などのファイルを利用する場合は、Colab へ対象のファイルをアップロードします。

- ① 左側のサイドバーのフォルダのアイコンをクリック
- ② 展開された「ファイル」に対象ファイルをドラッグ&ドロップで移動させます。

ライブラリのインストール

Colab ではよく利用されるライブラリは事前にインストールされています。
インストールされていないライブラリを使用する場合は、インストールが必要です。



```

import pytorch_lightning as pl

ModuleNotFoundError                                Traceback (most recent call last)
<ipython-input-12-77f7c7eb909b> in <module>()
----> 1 import pytorch_lightning as pl

ModuleNotFoundError: No module named 'pytorch_lightning'

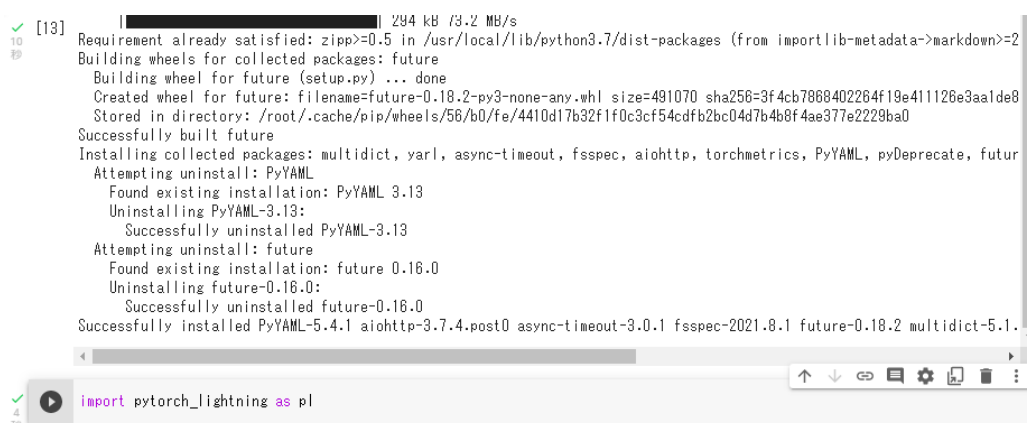
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
  
```

OPEN EXAMPLES SEARCH STACK OVERFLOW

- ・コードセルで `pip` コマンドを実行し、インストールする

例: `pip install pytorch_lightning`



```

[13] Requirement already satisfied: zipp>=0.5 in /usr/local/lib/python3.7/dist-packages (from importlib-metadata>=2
Building wheels for collected packages: future
Building wheel for future (setup.py) ... done
Created wheel for future: filename=future-0.18.2-py3-none-any.whl size=491070 sha256=3f4cb7868402264f19e411126e3aalde8
Stored in directory: /root/.cache/pip/wheels/56/b0/fe/4410d17b32f1f0c3cf54cdfb2bc04d7b4b8f4ae377e2229ba0
Successfully built future
Installing collected packages: multidict, yarl, async-timeout, fsspec, aiohttp, torchmetrics, PyYAML, pyDeprecate, futur
Attempting uninstall: PyYAML
Found existing installation: PyYAML 3.13
Uninstalling PyYAML-3.13:
Successfully uninstalled PyYAML-3.13
Attempting uninstall: future
Found existing installation: future 0.16.0
Uninstalling future-0.16.0:
Successfully uninstalled future-0.16.0
Successfully installed PyYAML-5.4.1 aiohttp-3.7.4.post0 async-timeout-3.0.1 fsspec-2021.8.1 future-0.18.2 multidict-5.1.
  
```

```

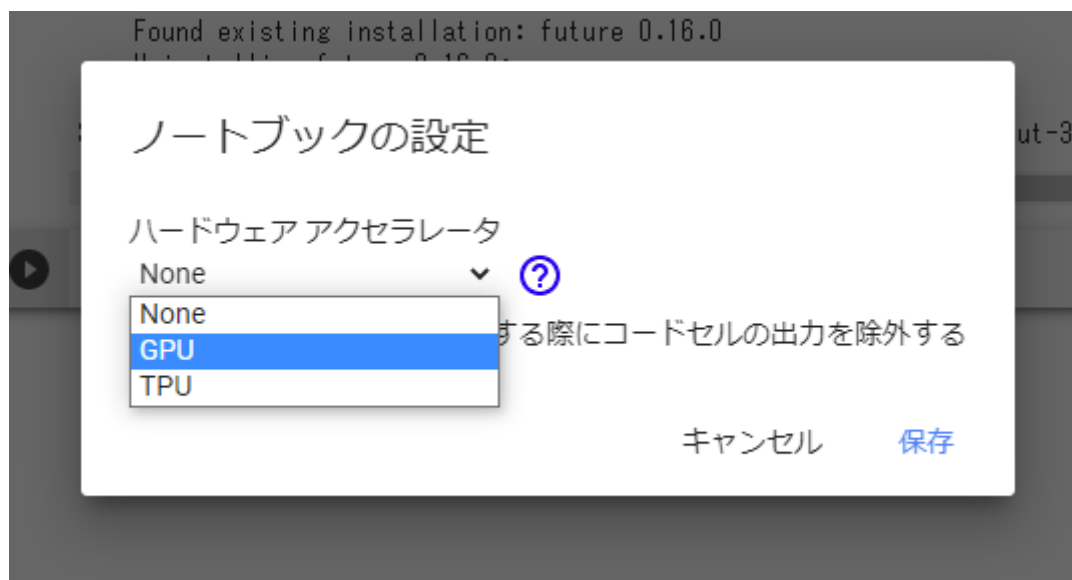
import pytorch_lightning as pl
  
```

GPU の利用

機械学習やディープランニングを実行する場合、大量のデータ処理を得意とした GPU を使用することがあります。

通常は GPU を使用するには特別な環境を自身で用意しなければなりませんが、Colab では無料で GPU を使用できます。

「ランタイム」→「ランタイムのタイプを変更」→「ハードウェアアクセラレータ」を「GPU」とすることで、GPU を利用できます。



接続が切れた場合

Colab では一定時間が経過すると、処理した結果がリセットされます。これを「ランタイムリセット」と言います。

ランタイムリセットには下記の 2 種類があります。

- 90 分 : 操作をしなくなってから 90 分経過
- 12 時間 : ノートブックを立ち上げてから 12 時間経過

ランタイムリセットされてしまった場合は、すべてのコードセルを上から順に実行すれば問題ありません。

すべてのコードセルをまとめて実行するには「ランタイム」→「再起動してすべてのセルを実行」とします。

Python 基礎

P-067

初版発行日：2019年11月06日
最終更新日：2021年12月17日
著 作：株式会社シンクスバンク
発 行 者：株式会社シンクスバンク



学習サービスのさらなる向上を。
ISO29993 認証取得。

KENスクールは、ISO（国際標準化機構）が学習サービスを提供する事業者に対する基本的な要求事項を定めた国際規格で、当該事業者が提供する学習サービスの品質を保証するものです。認証取得後も提供する学習サービスの品質の維持・改善に取り組み、継続的に向上させていくことに努めます。

本書の一部または全部を、株式会社シンクスバンクから正式な許諾を得ずに、いかなる方法（転載・転用・送信・上映等）においても無断で複写、複製することは禁止されています。